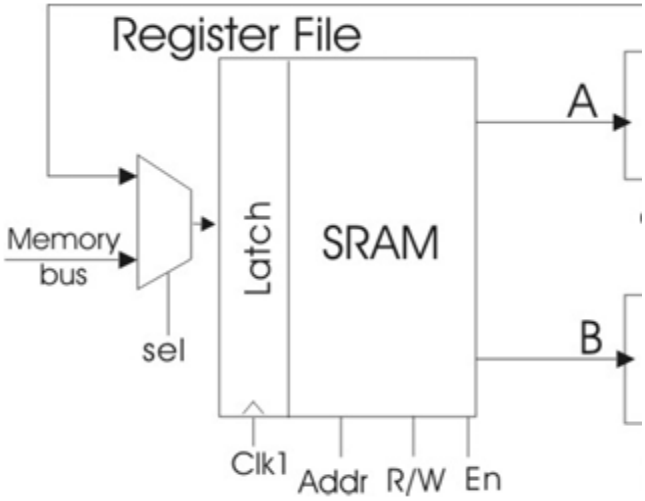
Final Project Report

SRAM



By

Nisan Alklany .

Mihde Khateeb .

Weaam Asaad .

Introduction to VLSI 2025B

7/1/2025

Contents

Project Overview: 4
List of Figures: 5
List of Tables: 6
6T_Cell: 8
Schematic 6T Cell: 9
Symbol 6T Cell: 10
Layout for 6T Cell: 11
read/write access circuit: 12
read/write access circuit Schematic: 12
read/write access circuit Symbol: 13
read/write access circuit Layout: 13
read/write access circuit Simulation: 14
Precharge Circuit: 14
Precharge Circuit Schematic: 15
Precharge Circuit Symbol: 16
Precharge Circuit Layout: 17
Sense Amplifer: 17
Sense Amplifer Symbol: 19
Sense Amplifer Layout: 19
Sense Amplifer Stimulus: 20
Sense Amplifer Result: 21
Buffers: 22
Inverting tri-state schematic: 22
Inverting tri-state Symbol: 22
Inverting tri-state Layout: 23
Inverting tri-state Stimulus: 23
Inverting tri-state Stimulus result: 24
8 bit output buffer schematic: 24
8 bit output buffer Symbol: 26
8 bit output buffer Layout: 26
8 bit input buffer Schematic: 27
8 bit input buffer Symbol: 28
8 bit input buffer Layout: 28

8 bit buffer simulation: 29
Tri-state buffer output explanation : 29
3 to 8 bit Decoder: 30
Decoder truth table: 30
NAND4 Gate Schematic: 31
NAND4 Gate Symbol: 31
NAND4 Gate Layout: 32
3 to 8 bit Decoder schematic: 32
3 to 8 bit Decoder Symbol: 33
3 to 8 Decoder Layout: 34
3 to 8 Decoder Stimulus: 35
Decoder Delay Check: 35
Bigger inverter schimatic : 36
Bigger inverter symbol : 36
Bigger inverter Layout: 37
3 to 8 Decoder Layout after inverter improvement: 38
SRAM integration: 38
Top Level SRAM 39
SRAM Symbol: 40
SRAM Layout: 41
SRAM Testbench: 41
Latch: 46
Latch schematic: 46
Latch Symbol: 46
Latch Layout: 47
1 bit Latch Stimulus: 47
Final Integration: 49
Final project integration latch with SRAM : 49
Final Integration Layout: 50
Final integration testbench: 50
Final result for one cell : 53
Timing: 53
write access delay: 53
read access delay: 54

bitline discharge time: 54
Output Rise and Fall time: 55
Conclusions: 56
Layout Area: 56
Metal Layers: 56
Number of Transistors: 56
Conclusion table : 57
Summary of Experience: 58
Time Required: 58
Feedback: 58

Project Overview:

The goal of this project was to design, simulate, and integrate a custom 8×8 SRAM array using full-custom VLSI techniques in 180nm CMOS technology. The work involved building all key memory components from the transistor level, including 6T memory cells, read/write access circuits, precharge units, sense amplifiers, decoders, buffers, and control logic. Each block was carefully sized and verified through schematic simulation and physical layout in Cadence Virtuoso.

The SRAM array was designed to support both read and write operations, with synchronized control via clock signals and external enable. A 3-to-8 decoder with global enable selects the active wordline, and tri-state buffers ensure safe data flow on shared lines. The datapath included clocked latches to store intermediate data during operation phases.

To verify the functionality, a complete testbench was developed in Spectre using realistic input stimuli based on timing diagrams. Simulations demonstrated successful storage and retrieval of data across all memory rows, validated through waveforms at each stage: from inputs and internal latches to output after the sense amplifiers and buffers.

The final design achieved a worst-case write delay of 326.63 ps and a worst-case read delay of 402.46 ps, demonstrating reliable high-speed operation. Overall, the project provided practical experience in transistor-level memory design, timing analysis, and full-chip integration using industry-standard VLSI tools.

List of Figures:

Figure 1-the theoretical constraints presented in Lecture 9 8
Figure 2- 6T Cell Schematic 9
Figure 3- 7T Cell Symbol 10
Figure 4 - 6T SRAM Cell Layout 11
Figure 5-read/write circuit Schematic: 12
Figure 6-read/write circuit Symbol 13
Figure 7-read/write access circuit Layout 14
Figure 8-read/write access circuit Simulation result 14
Figure 9-Precharge Circuit Schematic 15
Figure 10-Precharge Circuit Symbol 16
Figure 11-Precharge Circuit Layout 17
Figure 12-Sense Amplifer Schematic 18
Figure 13-Sense Amplifer Symbol: 19
Figure 14-Sense Amplifer Layout 20
Figure 15-Sense Amplifier Stimulus 20
Figure 16-Sense Amplifier Result 21
Figure 17- Inverting tri-state schematic 22
Figure 18-Inverting tri-state Symbol 22
Figure 19-tri-state Layout 23
Figure 20-Inverting tri-state Stimulus text 23
Figure 21-Inverting tri-state Stimulus result 24
Figure 22-8 bit output buffer schematic 25
Figure 23-8 bit output buffer Symbol 26
Figure 24-8 bit output buffer Layout 26
Figure 25- 8 bit input buffer Schematic 27
Figure 26-8 bit input buffer Symbol 28
Figure 27-8 bit input buffer Layout 28
Figure 28-NAND4 Gate Schematic 31

Figure 29-NAND4 Gate Symbol 32
Figure 30-NAND4 Gate Layout 32
Figure 31 - 3 to 8 bit Decoder schematic 33
Figure 32 - 3 to 8 bit Decoder Symbol 34
Figure 33-3 to 8 Decoder Layout 34
Figure 34 - 3 to 8 Decoder Waves 35
Figure 35-decoder rise time with minimum inverter size 35
Figure 37-Bigger inverter schematic 36
Figure 38-Bigger inverter symbol 36
Figure 39-Bigger inverter Layout 37
Figure 41-WL rise time big inverter 37
Figure 42-3 to 8 Decoder Layout after inverter improvement 38
Figure 43-Top Level SRAM Schematic 39
Figure 44- SRAM Symbol 40
Figure 45-SRAM Layout 41
Figure 46-SRAM Stimulus file 42
Figure 47- part 1 of SRAM simulation output decoder ADDRESS 42
Figure 48 - part 2 SRAM read/write and Data inputs 43
Figure 49- part 3 SRAM read/write Data inputs after input buffer 44
Figure 50-part 3 SRAM Data outputs after sense amplifer 44
Figure 51-part 3 SRAM read/write Data outputs after buffer 45
Figure 52- Latch schematic 46
Figure 53-Latch Symbol 46
Figure 54-Latch Layout 47
Figure 55-Latch Stimulus 47
Figure 56-Latch Stimulus result 48
Figure 57-Final project integration Latch with SRAM 49
Figure 58-Final Integration Layout SRAM +8bit Latch 50
Figure 59-Final Integration stimulus 51
Figure 60-Timing diagram for a complete 2-cycle data path function from project description. 52
Figure 61-Timing diagram for data path function wave 52
Figure 62- summary result for one cell 53
Figure 63-Write Access Propagation Delay High to Low & Low to High 54
Figure 64-Read Access Low to High Propagation Delay 54
Figure 65 - Read Access High to Low Propagation Delay 54
Figure 66-bitline discharge time 55
Figure 67-output fall time 55
Figure 68-output rise time 55

List of Tables:

Table 1- 3 to8 bit decoder truth table 30
Table 2-Timing table 55
Table 3-size conclusion table 57
Table 4-timing conclusion table 57

Design and Results:

6T Cell:

We began our design by implementing a 6T SRAM cell . To ensure reliable operation, we carefully sized the transistors based on the theoretical constraints presented in Lecture 9, particularly the required **Cell Ratio (CR=2)** and **Pull-up Ratio (PR=0.5)**. The selected widths were **W4 = 1.6 µm** (pull-down NMOS),
W5 = 800 nm (access NMOS), and **W6 = 400 nm** (pull-up PMOS), ensuring the condition

This sizing guarantees stable read and write operations, balancing drive strength and noise margins as shown in the figure below.

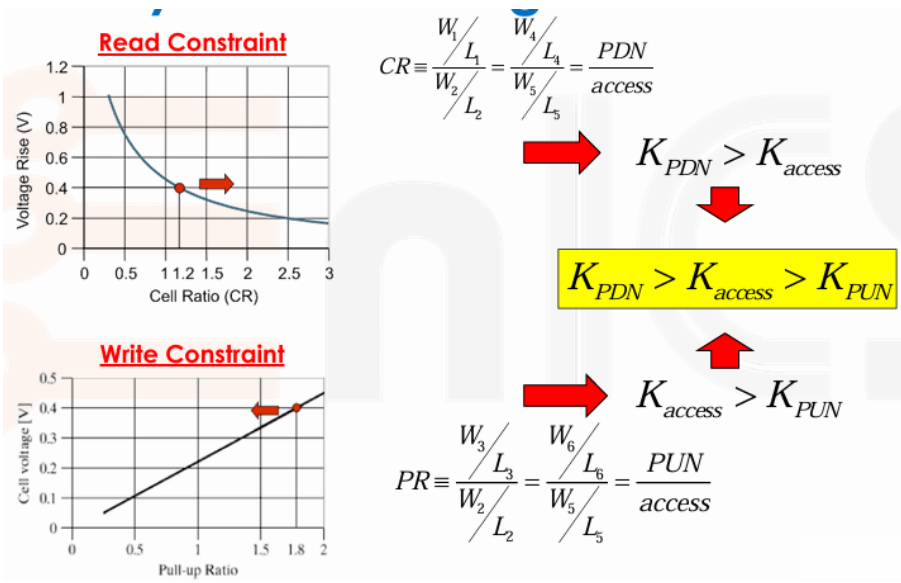


Figure 1-the theoretical constraints presented in Lecture 9

Schematic 6T Cell:

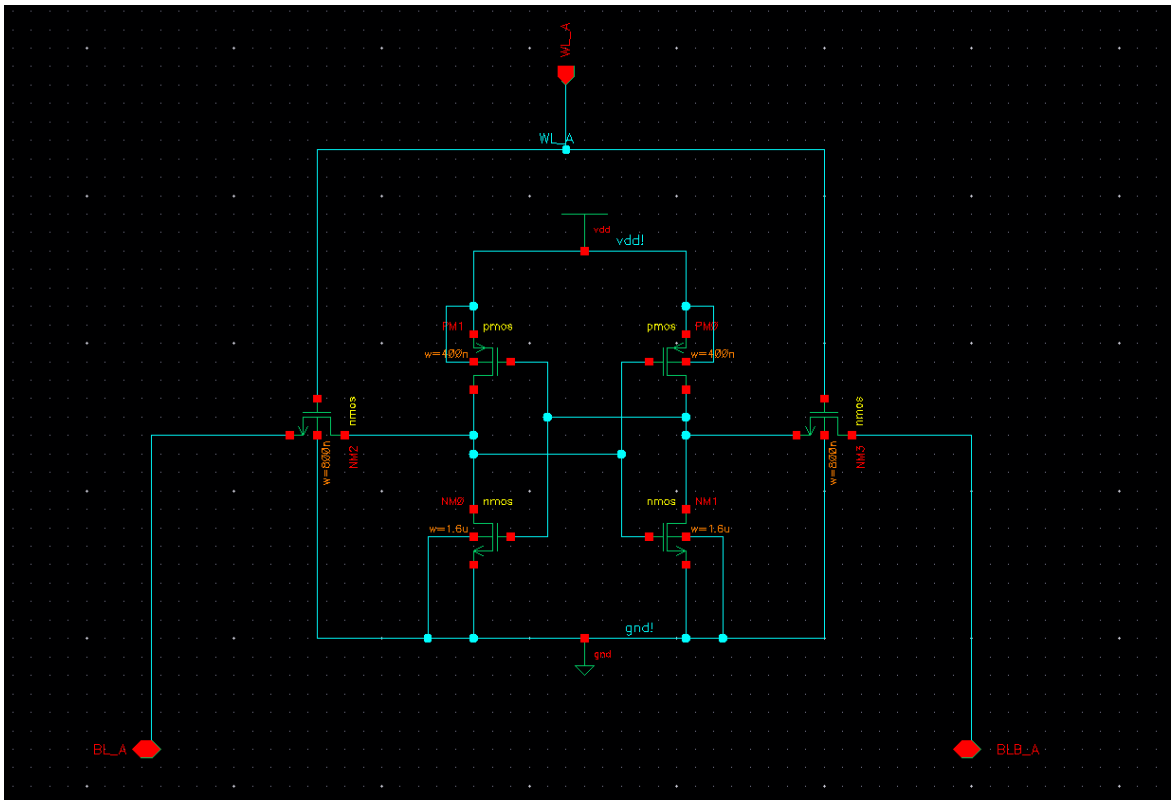


Figure 2- 6T Cell Schematic

Symbol 6T Cell:

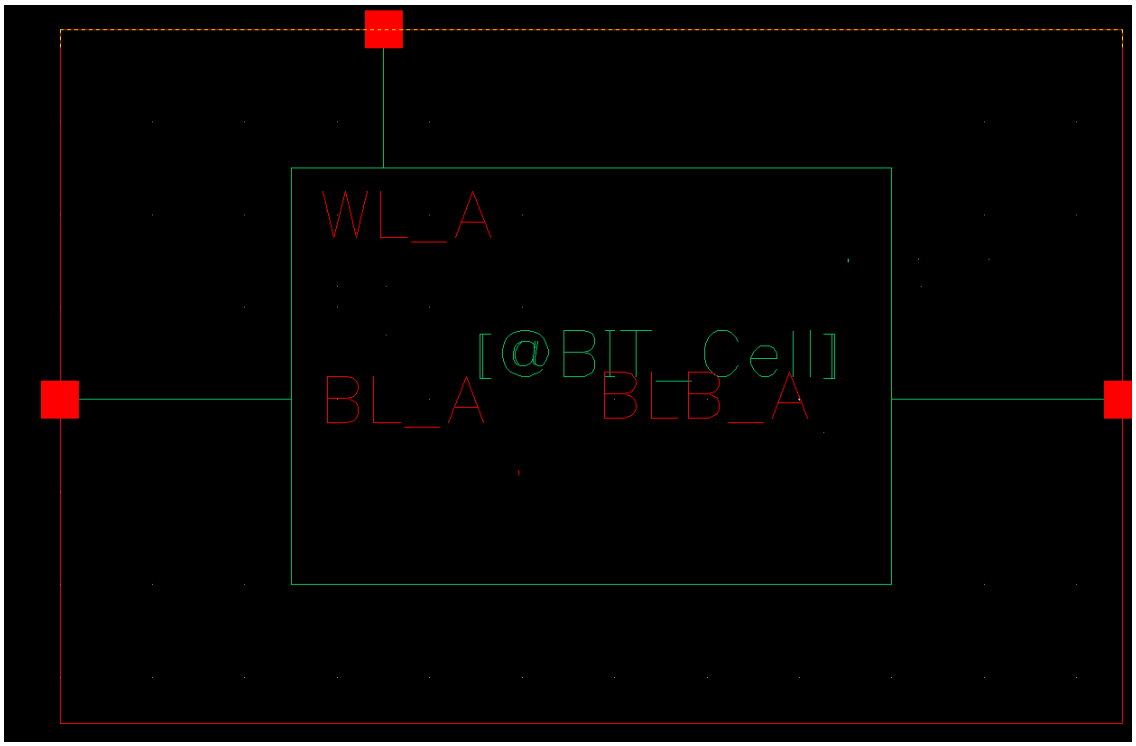


Figure 3- 7T Cell Symbol

Layout for 6T Cell:

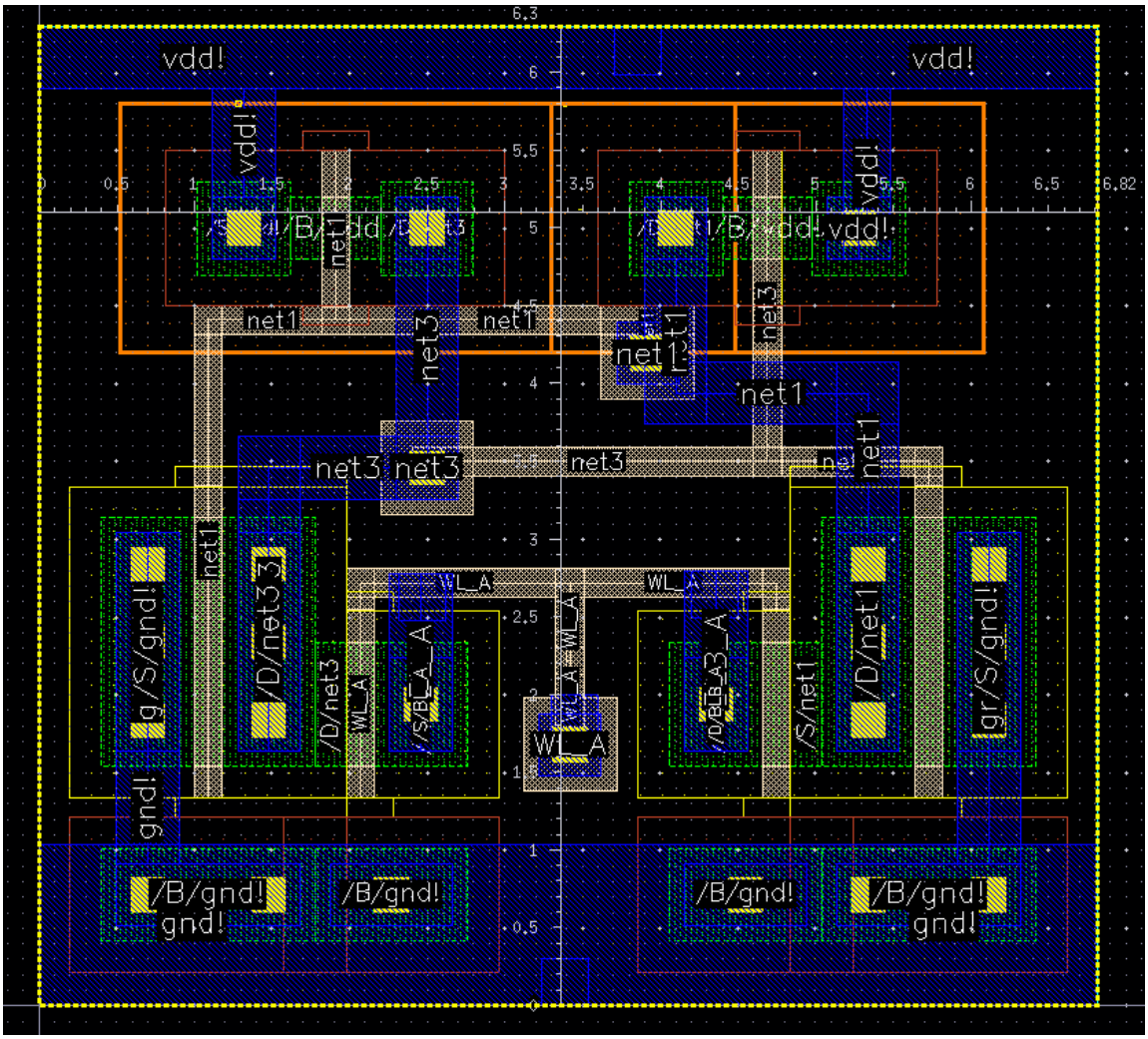


Figure 4 - 6T SRAM Cell Layout

read/write access circuit:

After implementing the core 6T storage element, we designed the read/write access circuit to handle memory operations. The cell contains access transistors controlled by the word line (WR), which enables read or write access to the internal node A . During **write operations**, the bitlines (BL, BLB) are driven externally to force the required value into the cell; if the cell initially stores a '1', forcing BL=0 and activating WR will flip the internal node to store '0'. If the cell holds '0', one side of the cell will slightly discharge its corresponding bitline, and this voltage difference is detected. The transistor sizing ensures that read and write operations behave correctly without corrupting the stored data. The schematic below shows the implementation of the read/write path integrated with the cell's feedback loop.

read/write access circuit Schematic:

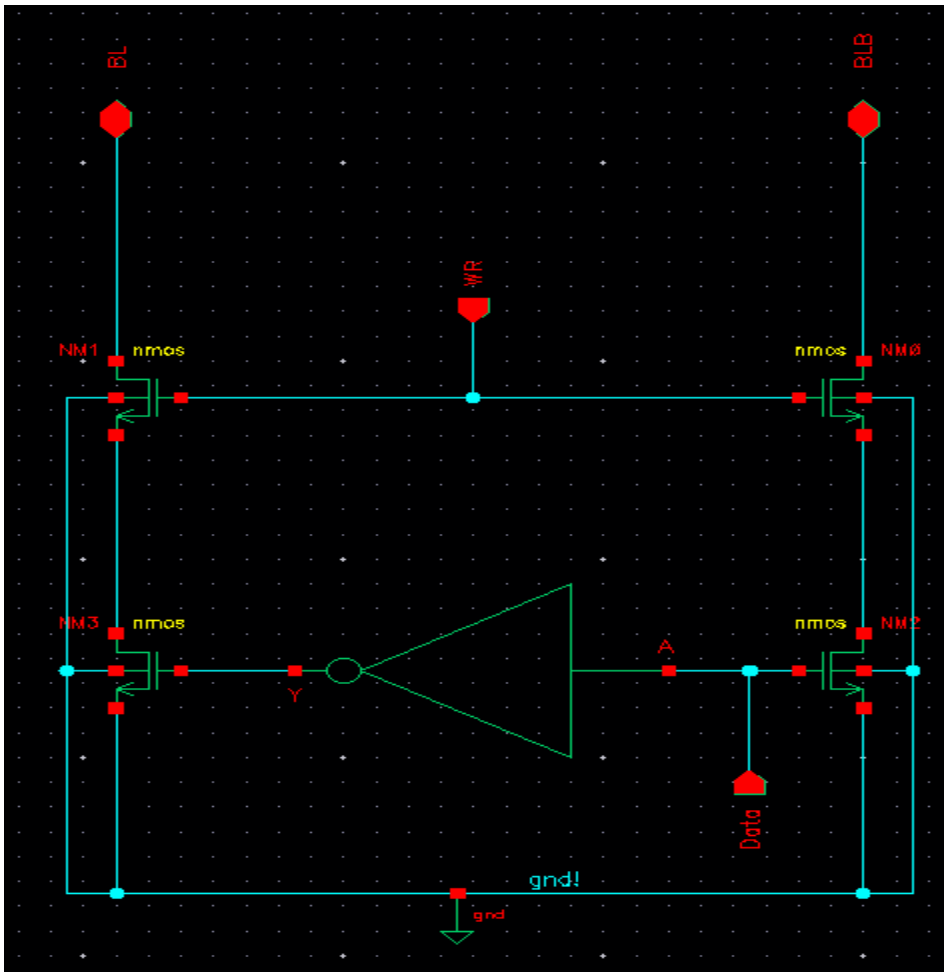


Figure 5-read/write circuit Schematic:

read/write access circuit Symbol:

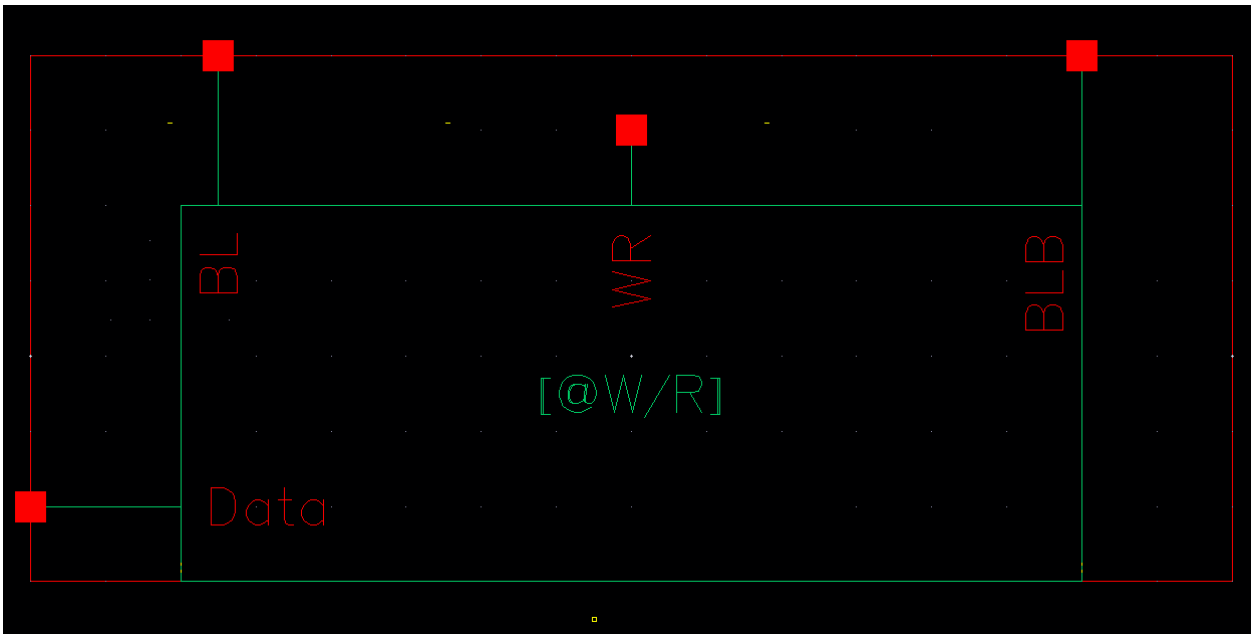


Figure 6-read/write circuit Symbol

read/write access circuit Layout:

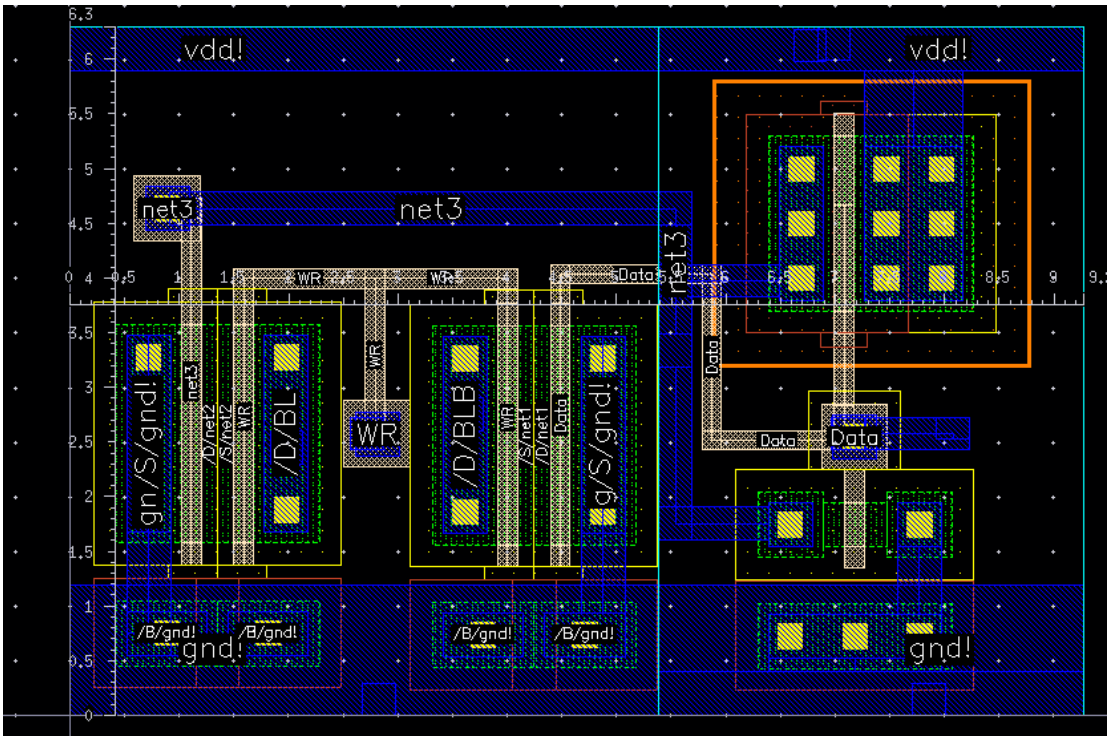


Figure 7-read/write access circuit Layout

read/write access circuit Simulation:

Net 3 corresponds to BL and net 17 corresponds to BLB. The signal in0 represents the input data. We observe that when the input is low, BL is pulled low, and when the input is high, BLB is pulled low. This confirms that the data is correctly written and stored in the SRAM.

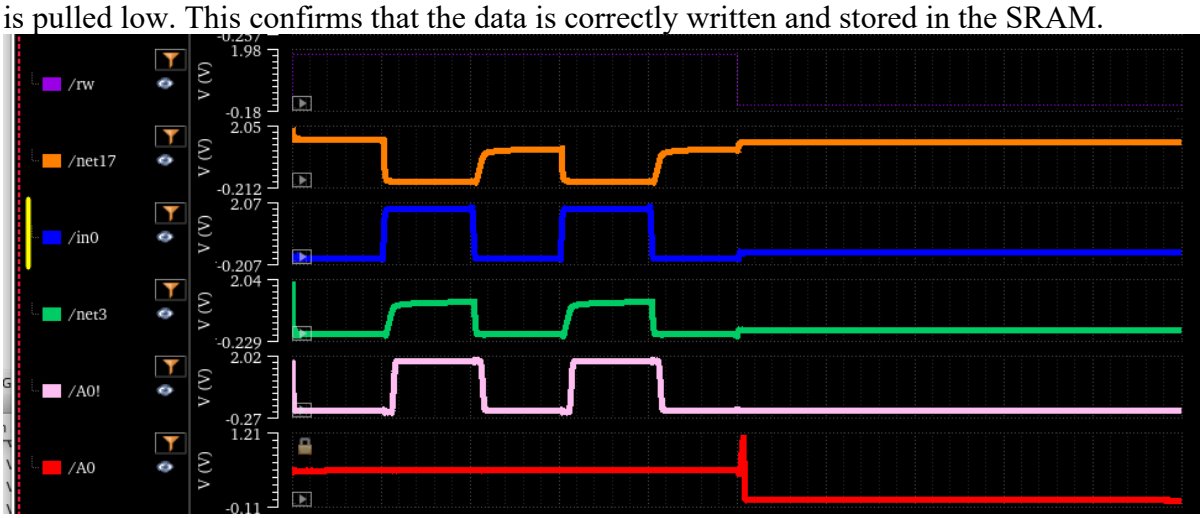


Figure 8-read/write access circuit Simulation result

Precharge Circuit:

We added a precharge circuit to ensure the bitlines (BL and BLB) are set to a known state before each read enable operation. The circuit uses PMOS transistors to pull both bitlines up to VDD when the **Pre** signal is active. This step is crucial for enabling the sense amplifier to detect small

voltage differences during a read. The precharge is enabled just before a read cycle and then disabled to allow the memory cell to affect the bitlines.

To ensure reliable operation, the **precharge PMOS transistors are sized slightly smaller(400n) than the access NMOS transistors**, so that during a read, the access path can successfully discharge one bitline without being overpowered by the precharge network. This sizing balance improves read stability and prevents destructive reads.

Precharge Circuit Schematic:

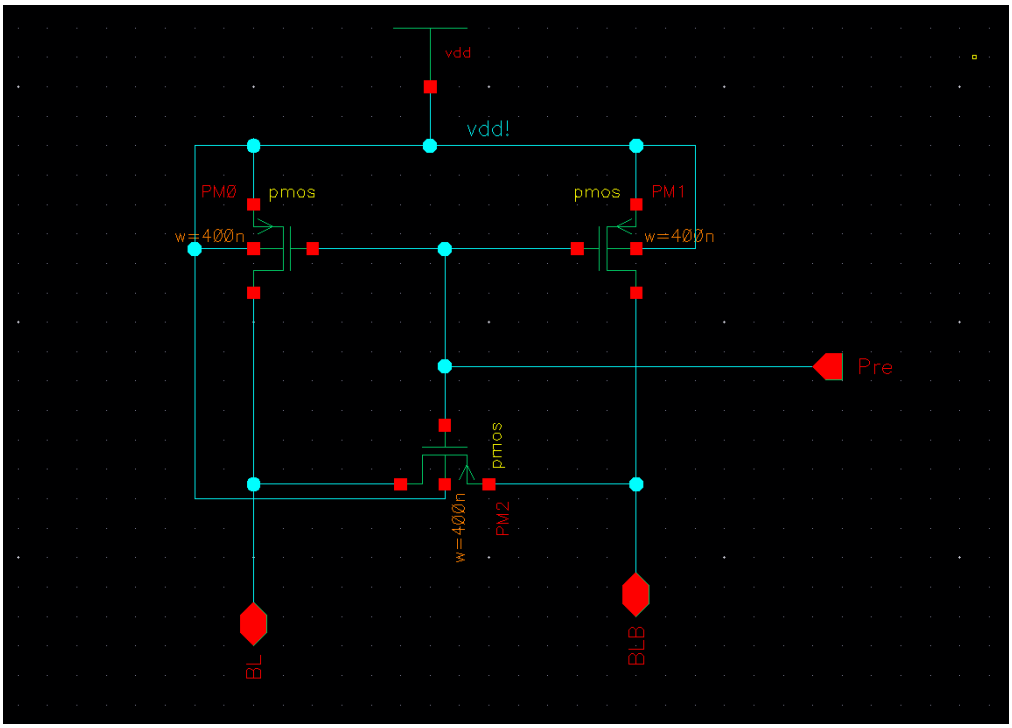


Figure 9-Precharge Circuit Schematic

Precharge Circuit Symbol:

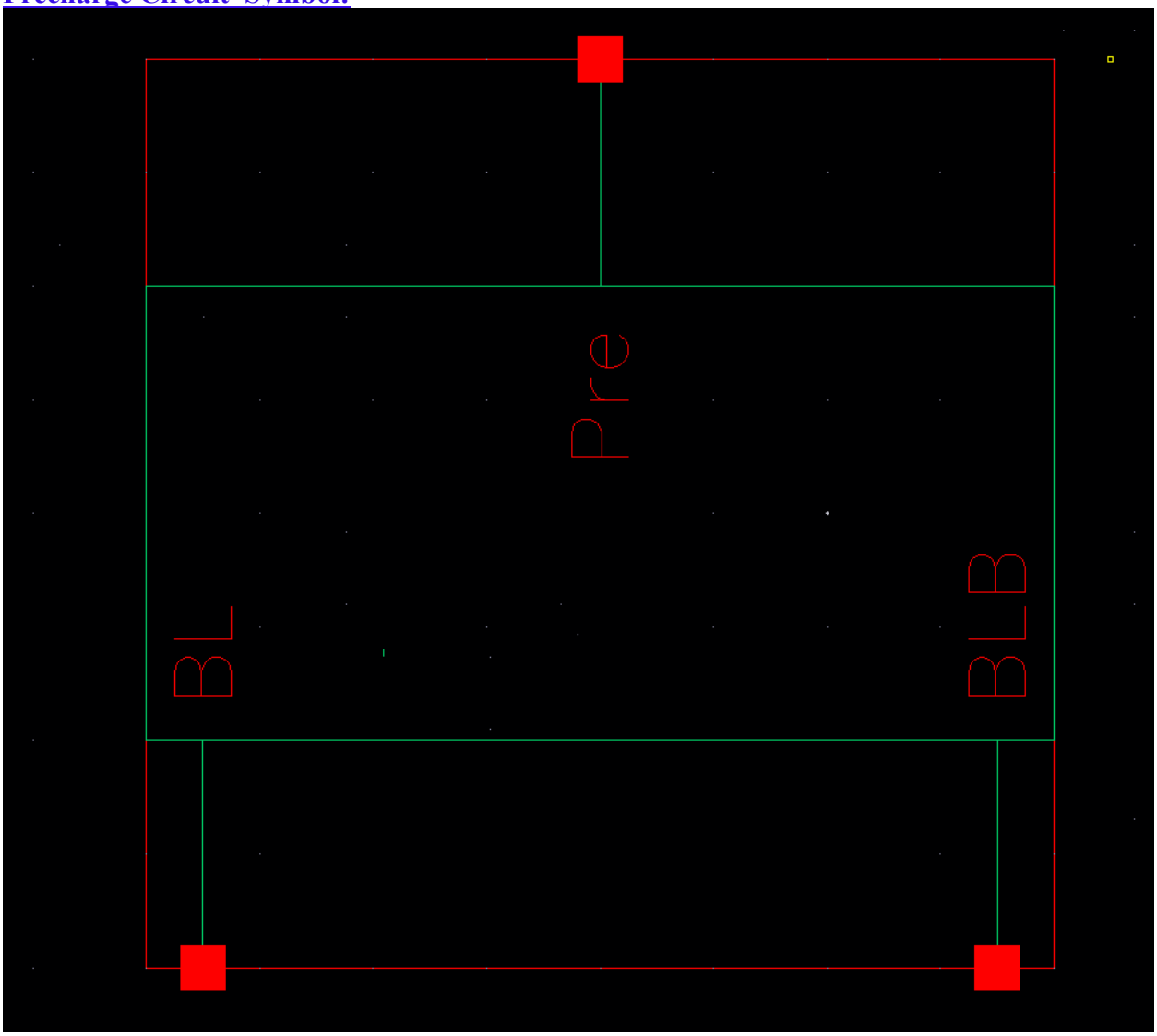


Figure 10-Precharge Circuit Symbol

Precharge Circuit Layout:

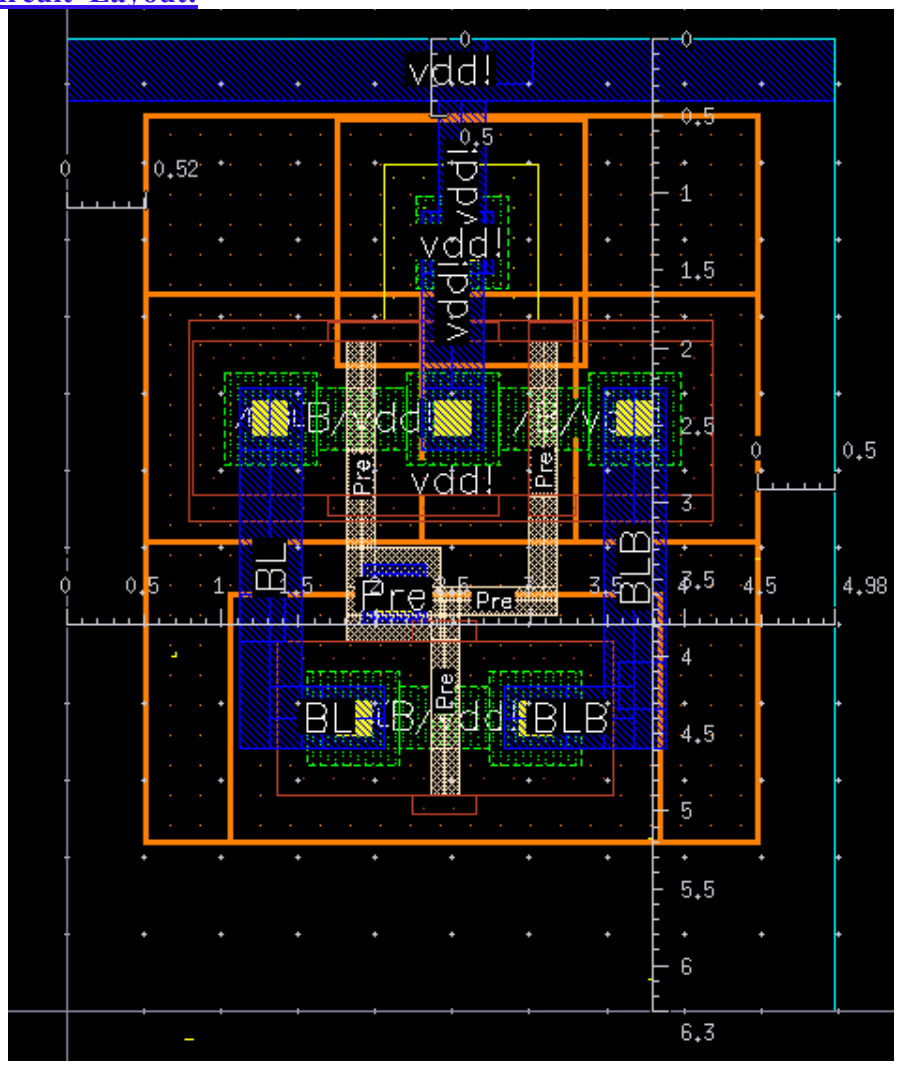


Figure 11-Precharge Circuit Layout

Sense Amplifer:

We designed a differential sense amplifier to detect small voltage differences between the SRAM bitlines (BL and BLB) during read cycles. The amplifier uses two NMOS transistors connected to BL and BLB as differential inputs, and two PMOS transistors as active loads. When the enable signal activates the circuit, the side with the lower bitline voltage discharges faster, and the output node resolves to a full logic level. To ensure balanced and symmetric switching behavior, we sized the transistors based on design guidelines from the lecture: the PMOS width

was chosen a the CMOS width (i.e., Wp=2Wn) providing proper current drive and gain for fast and reliable sensing.

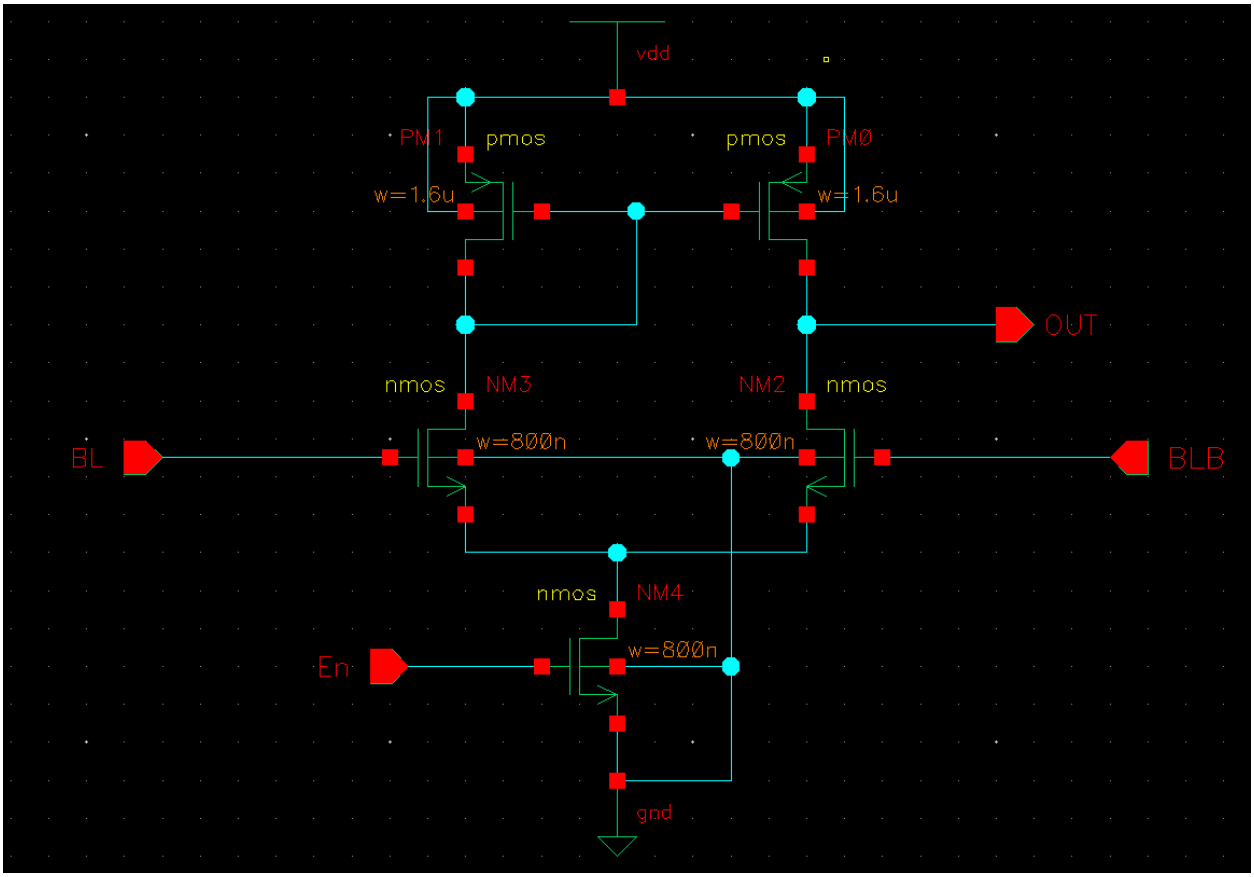


Figure 12-Sense Amplifer Schematic

Sense Amplifer Symbol:

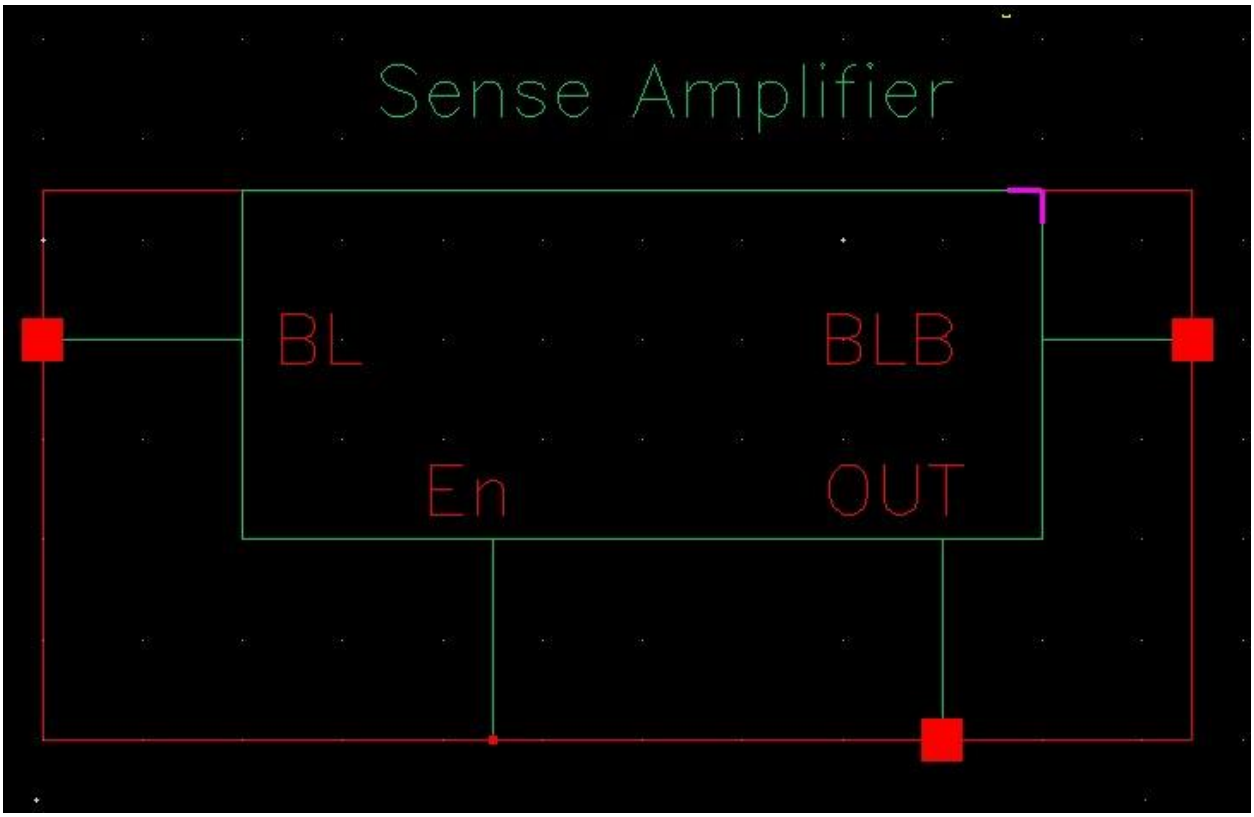


Figure 13-Sense Amplifer Symbol:

Sense Amplifer Layout:

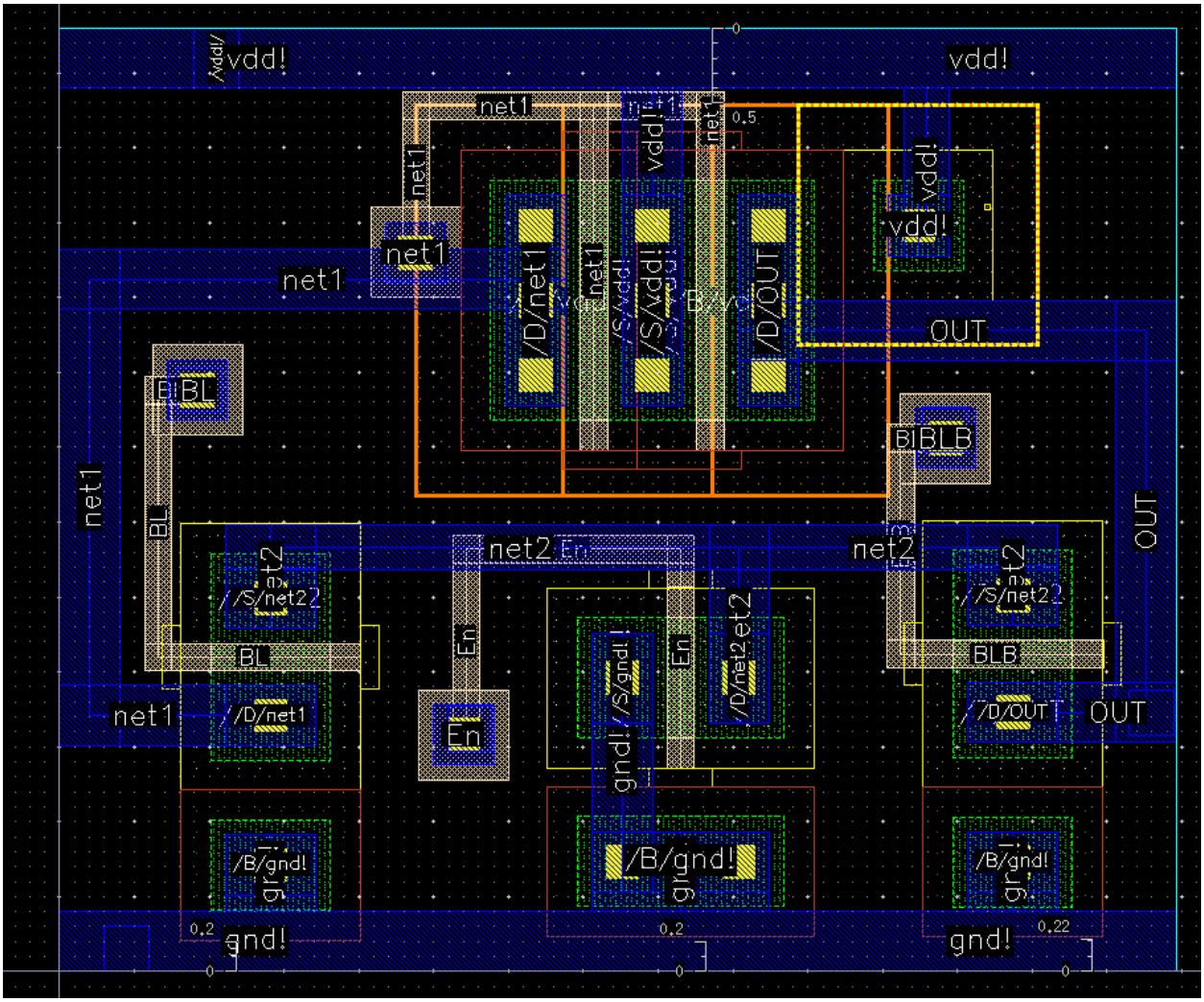


Figure 14-Sense Amplifer Layout

Sense Amplifer Stimulus:

```
simulator lang=spectre
global gnd!
vdd (vdd! 0) vsource dc=1.8
gnd (gnd! 0) vsource dc=0
V1 (BL 0) vsource type=pulse val0=0 val1=1.4 delay=0n rise=0.05n fall=0.05n width=5n period=10n
V2 (BLB 0) vsource type=pulse val0=0 val1=1.4 delay=5n rise=0.05n fall=0.05n width=5n period=10n
V4(En 0) vsource type=pwl wave=\[0n 0 0.05n 1.8 50n 1.8]
```

Figure 15-Sense Amplifier Stimulus

Sense Amplifer Result:

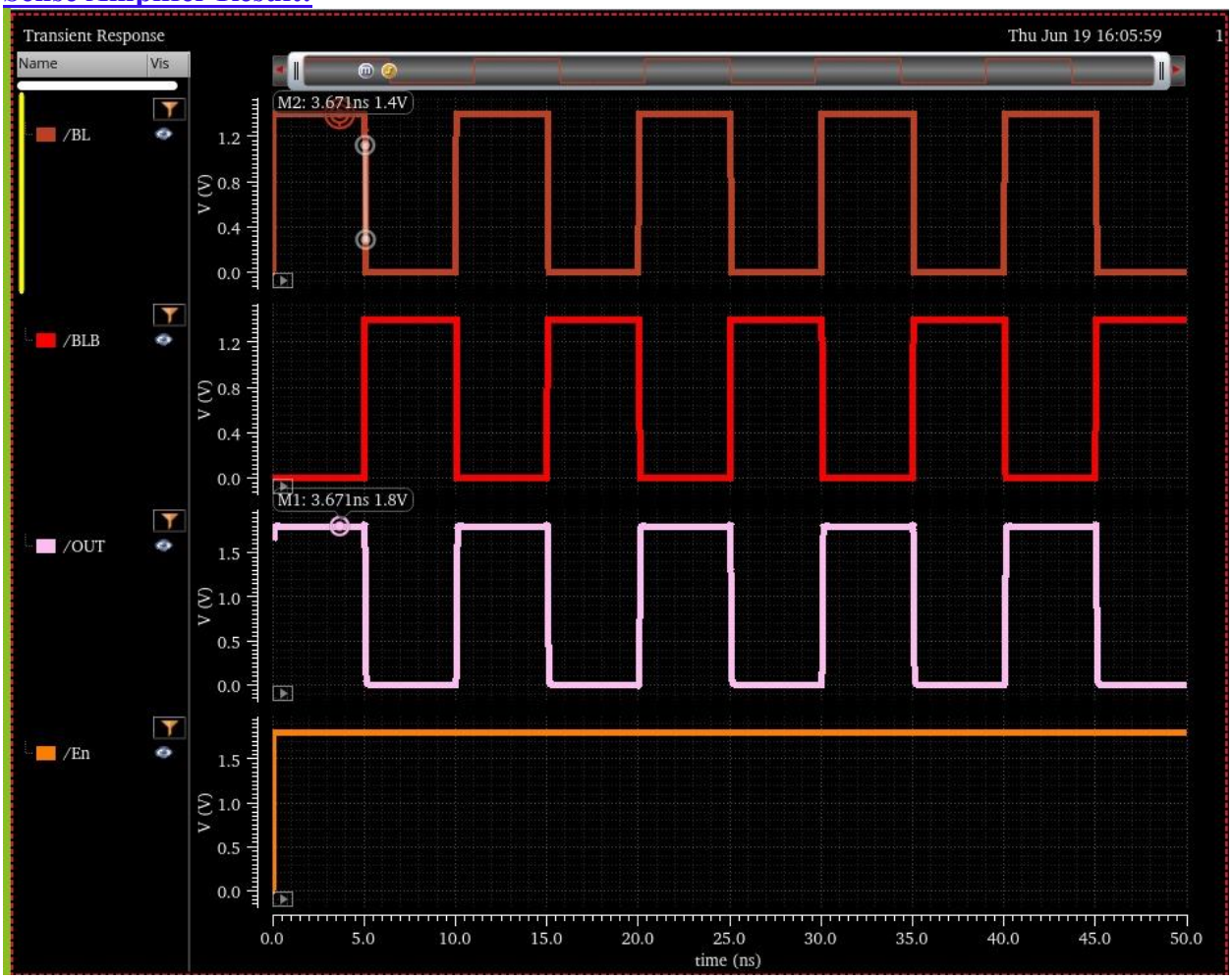


Figure 16-Sense Amplifier Result

The Sense amplifer detect small voltage difference and amplife the low (1.4v) voltage in BL to vdd(1.8v)

Buffers:

Inverting tri-state schematic:

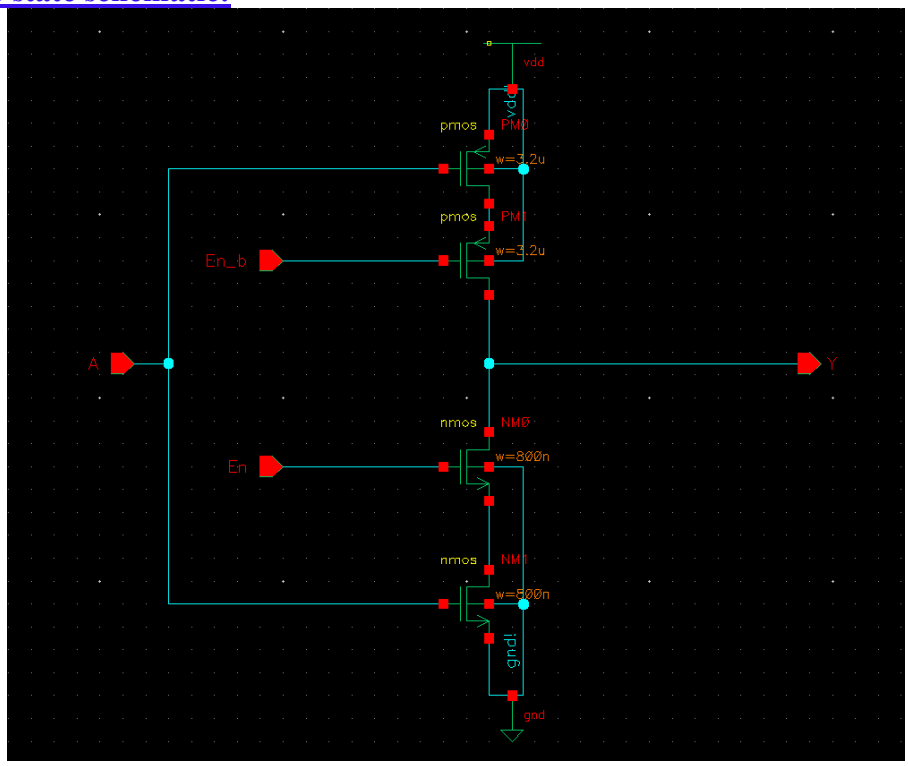


Figure 17- Inverting tri-state schematic

Inverting tri-state Symbol:

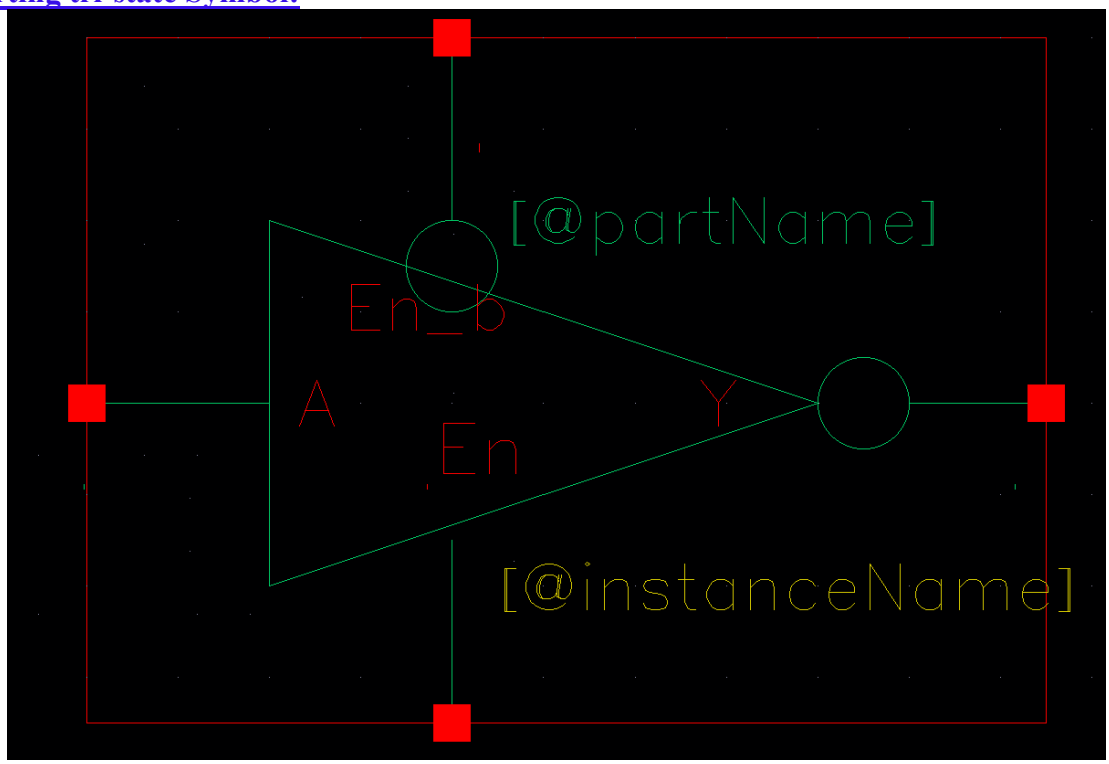


Figure 18-Inverting tri-state Symbol

Inverting tri-state Layout:

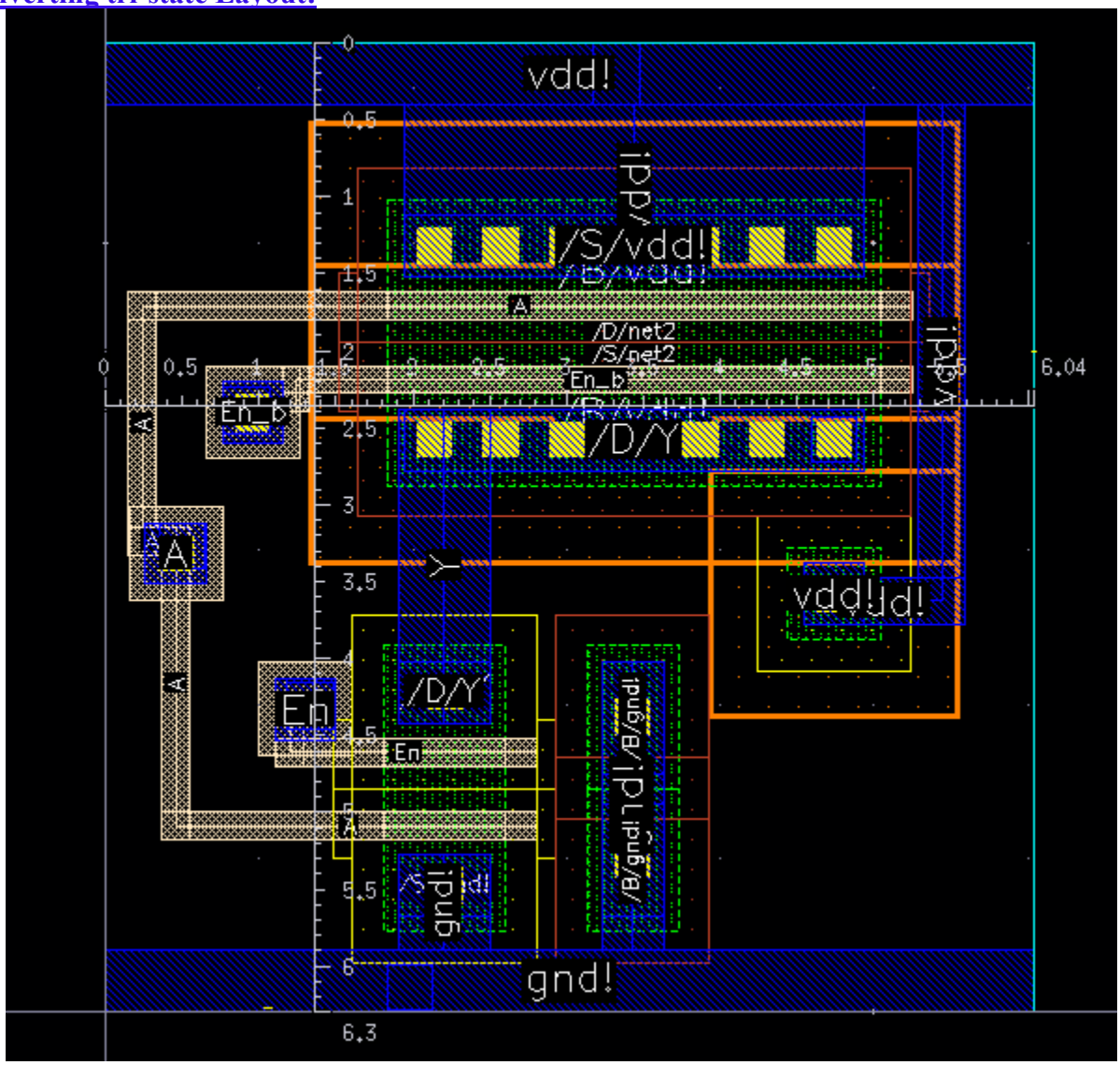


Figure 19-tri-state Layout

Inverting tri-state Stimulus:

```
simulator lang=spectre
global gnd!
vdd (vdd! 0) vsource dc=1.8
gnd (gnd! 0) vsource dc=0
V0 (A 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=5n period=10n
V1 (En 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=25n period=50n
V2 (En_b 0) vsource type=pulse val0=1.8 val1=0 delay=0n rise=0.05n fall=0.05n width=25n period=50n
```

Figure 20-Inverting tri-state Stimulus text

Inverting tri-state Stimulus result:

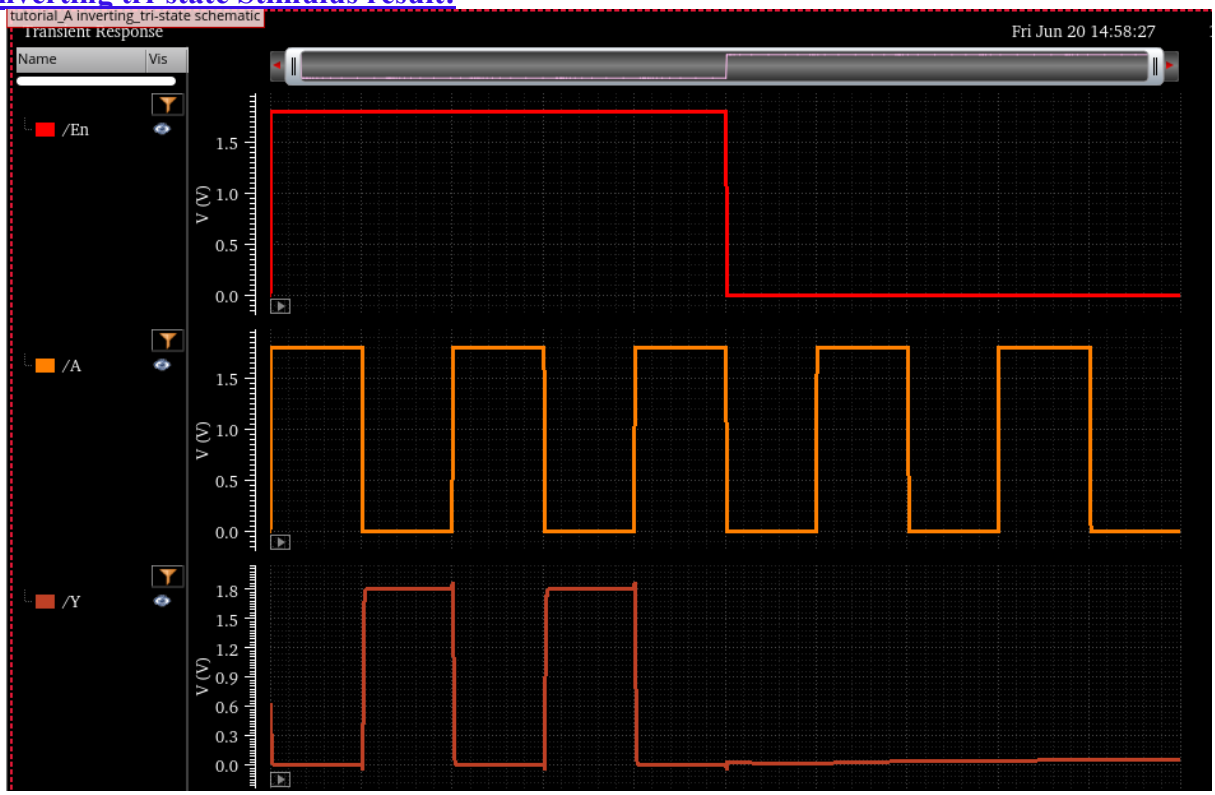


Figure 21-Inverting tri-state Stimulus result

The simulation shows the operation of a tri-state inverter buffer. When /En is high, the buffer output Y actively drives the inverse of the input A. When /En goes low, the output enters a high-impedance (High-Z) state, disconnecting from the circuit. This behavior is essential for bus sharing and safe control of multi-driver lines.

8 bit output buffer schematic:

Both the 8-bit input and 8-bit output buffers were constructed using an array of identical 1-bit buffer cells. While the core buffer structure is shared, the enable logic differs between the two:
The input buffer is enabled by the wr (write enable) signal and enable (rwen) signal, allowing data to enter the SRAM only when a write is requested .
The output buffer is controlled by the enable signal and also read signal, allowing data to leave the SRAM only during a valid read phase.
This modular design simplifies layout, promotes reusability, and ensures clean, synchronized data transfer across the SRAM interface .

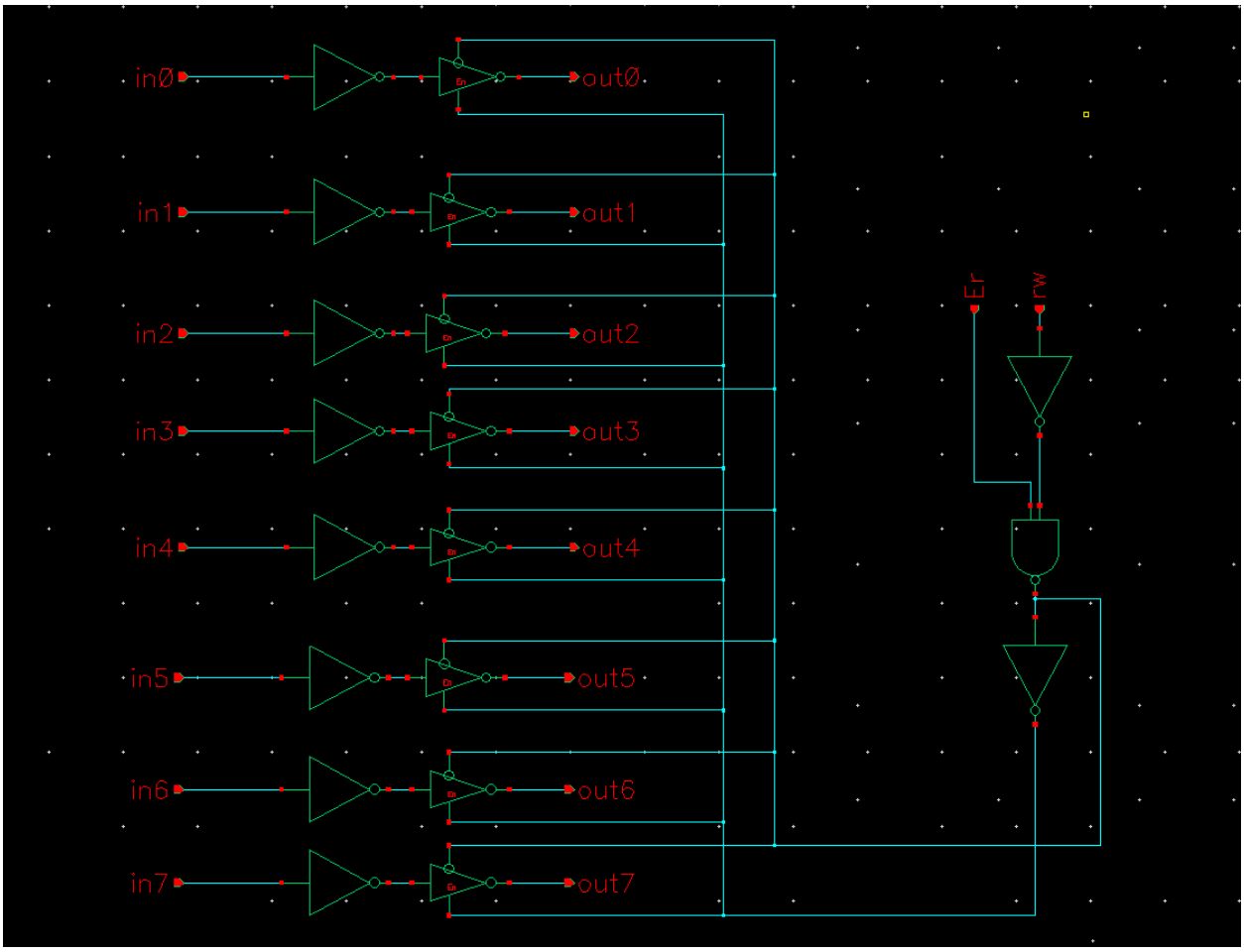


Figure 22-8 bit output buffer schematic

8 bit output buffer Symbol:

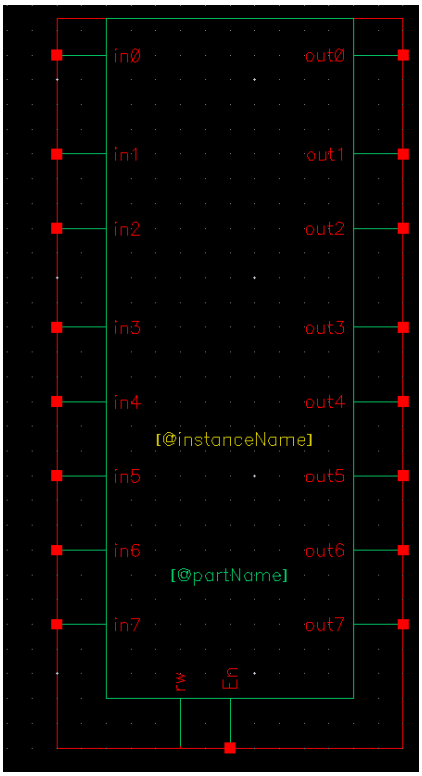


Figure 23-8 bit output buffer Symbol

8 bit output buffer Layout:

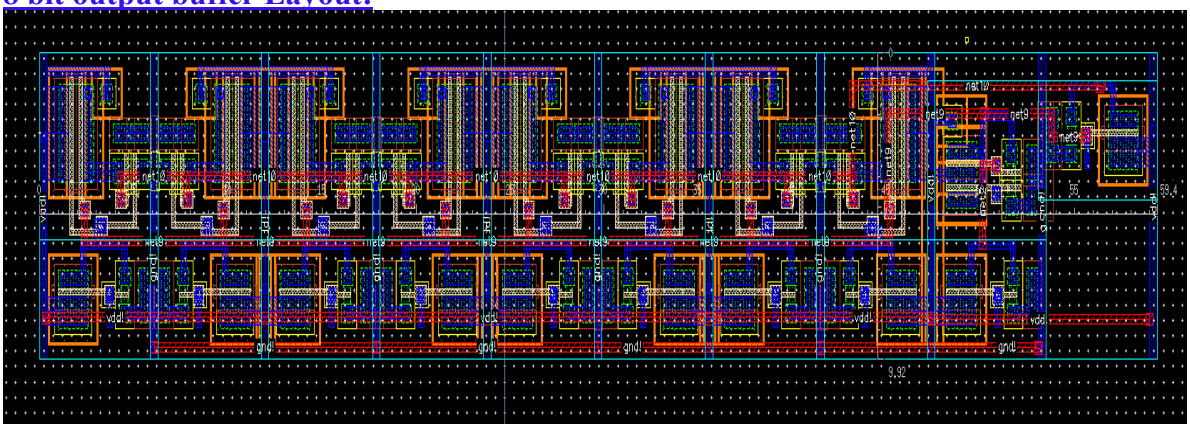


Figure 24-8 bit output buffer Layout

8 bit input buffer Schematic:

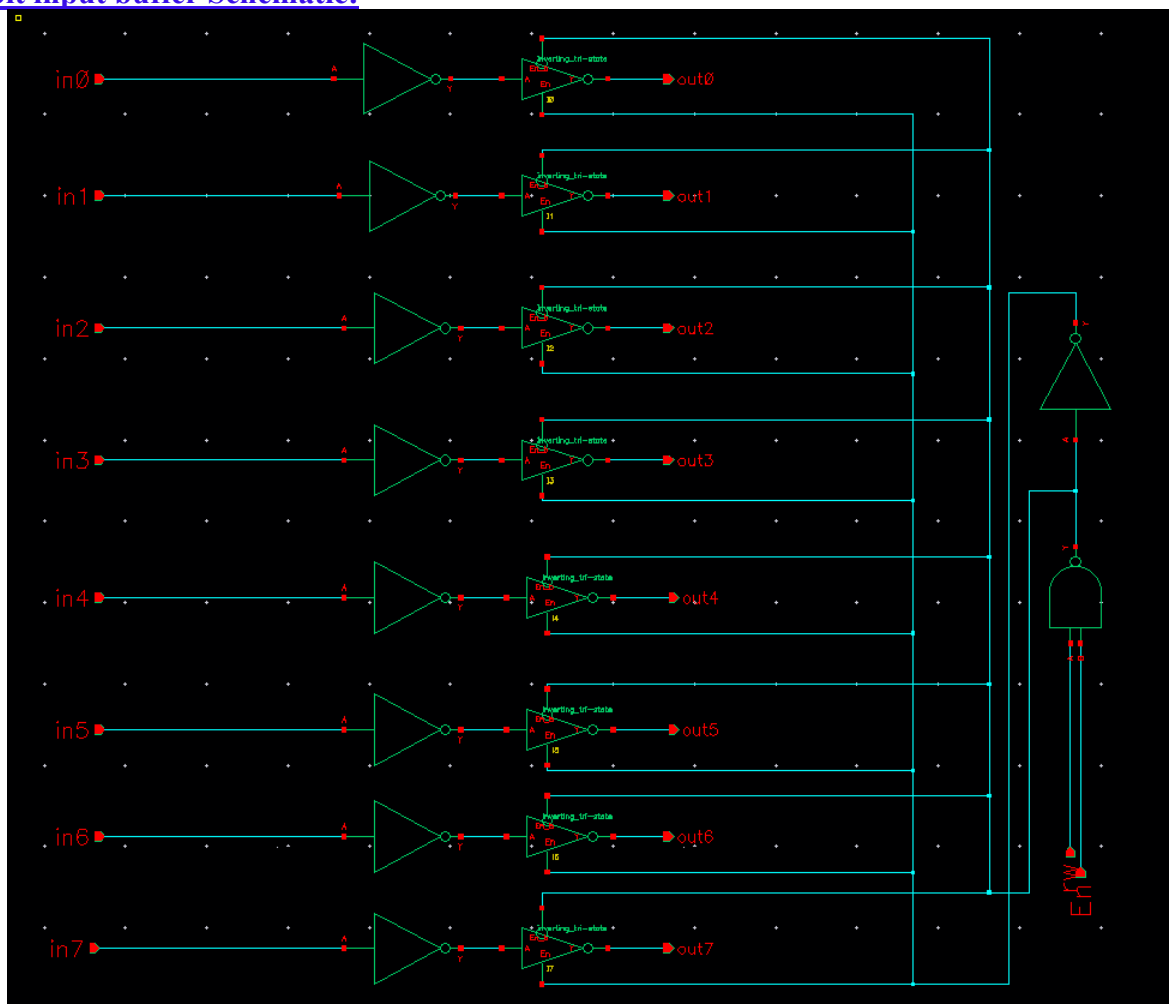


Figure 25- 8 bit input buffer Schematic

8 bit input buffer Symbol:

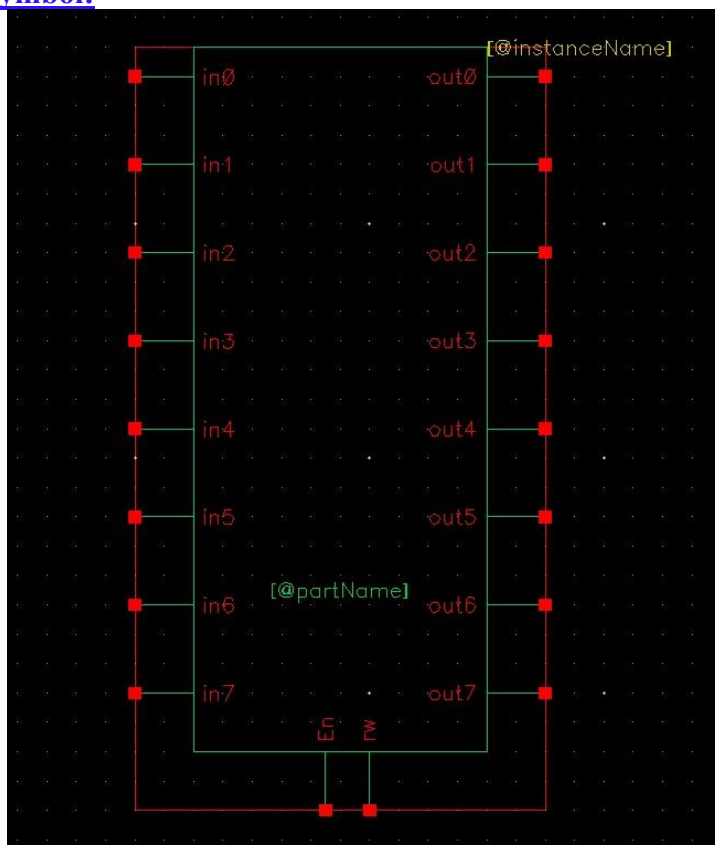


Figure 26-8 bit input buffer Symbol

8 bit input buffer Layout:

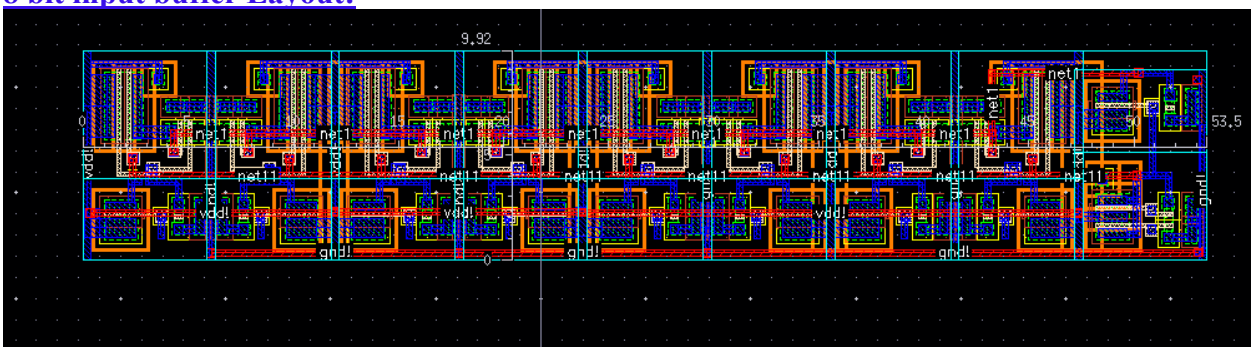
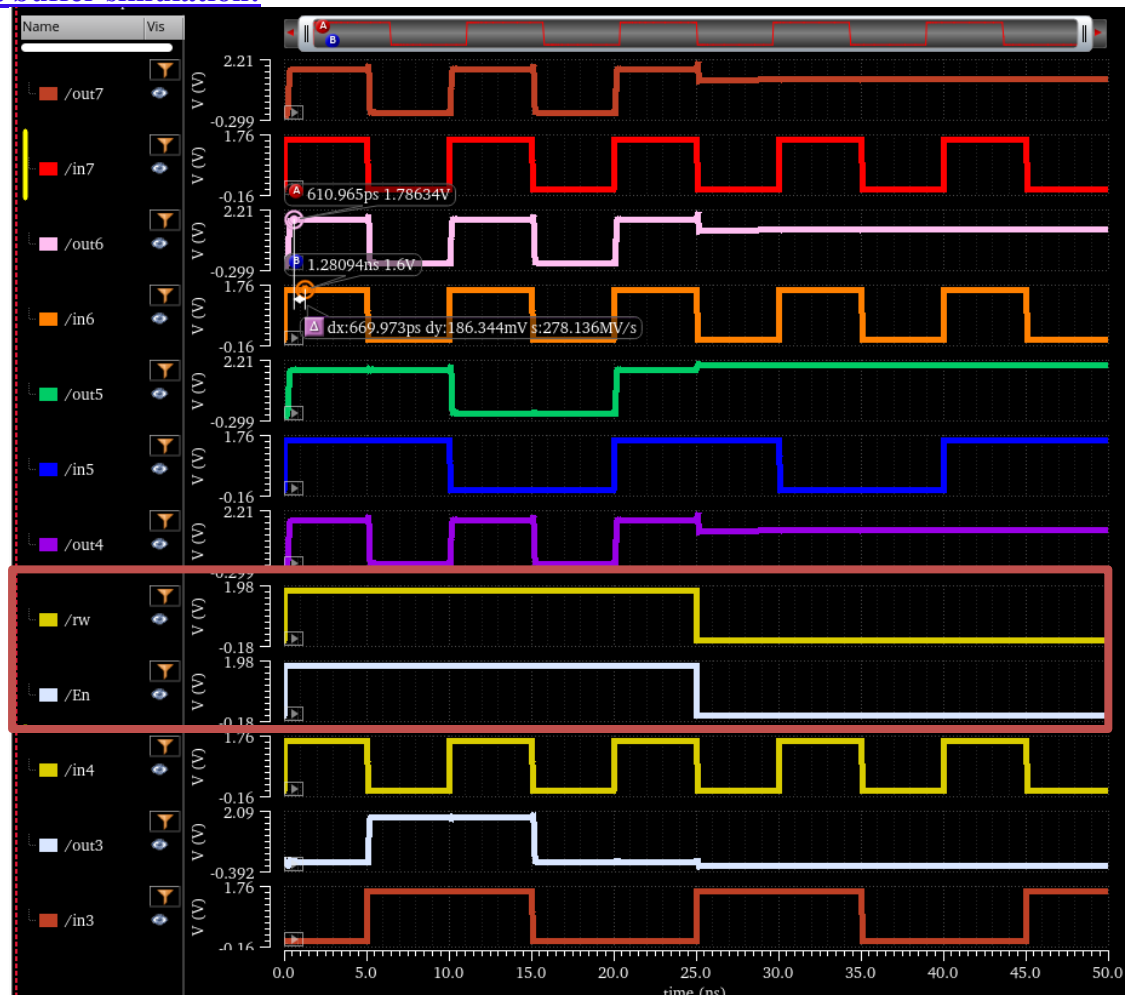


Figure 27-8 bit input buffer Layout

8 bit buffer simulation:



Tri-state buffer output explanation :

We observe that when the enable signal (/En) is high, the tri-state buffer passes the input signal to the output. The input voltage, which is around 1.6V, is amplified to the full logic level (VDD at the output. This indicates the buffer is actively driving the line.
When the enable signal is low, the buffer enters high-impedance mode (Hi-Z). In this state, the output is not driven by the buffer and becomes electrically floating. The actual voltage observed at the output in this case is not a valid logic level and depends on external factors such as parasitic capacitance, leakage currents, and the previous state of the wire.
This explains why we still see a voltage (e.g., 2.1V) at the output even though the buffer is in Hi-Z — this is a residual charge, not an actively driven signal.

The input buffer is enabled when both En and rw are high (write with enable).

The output buffer is enabled when En is high and rw is low(read with enable).

3 to 8 bit Decoder:

To address the SRAM rows, we implemented a 3-to-8 decoder using a 4-input NAND gate structure. The three address inputs (`A<0:2>`) define the selected wordline, while the fourth input serves as an **enable signal(<rfen>)**. This allows global control over the decoder activation, preventing unintended access during idle or disabled states. The outputs of the NAND gate are inverted to create active-high word lines, enabling proper connection to the SRAM wordline drivers. This method simplifies control logic and ensures that only one row is selected when the enable is active.

Decoder truth table:

A2	A1	A0	W0	W1	W2	W3	W4	W5	W6	W7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Table 1- 3 to8 bit decoder truth table

To implement the decoder, we initially chose to design a 4-input NAND gate—three inputs for address selection and one for an enable signal.

This choice was made to reduce the total number of transistors compared to using multiple 2-input NAND gates and inverters . Transistor sizing was optimized based on the theory presented in the lecture, ensuring proper logic levels and switching performance while minimizing area.

NAND4 Gate Schematic:

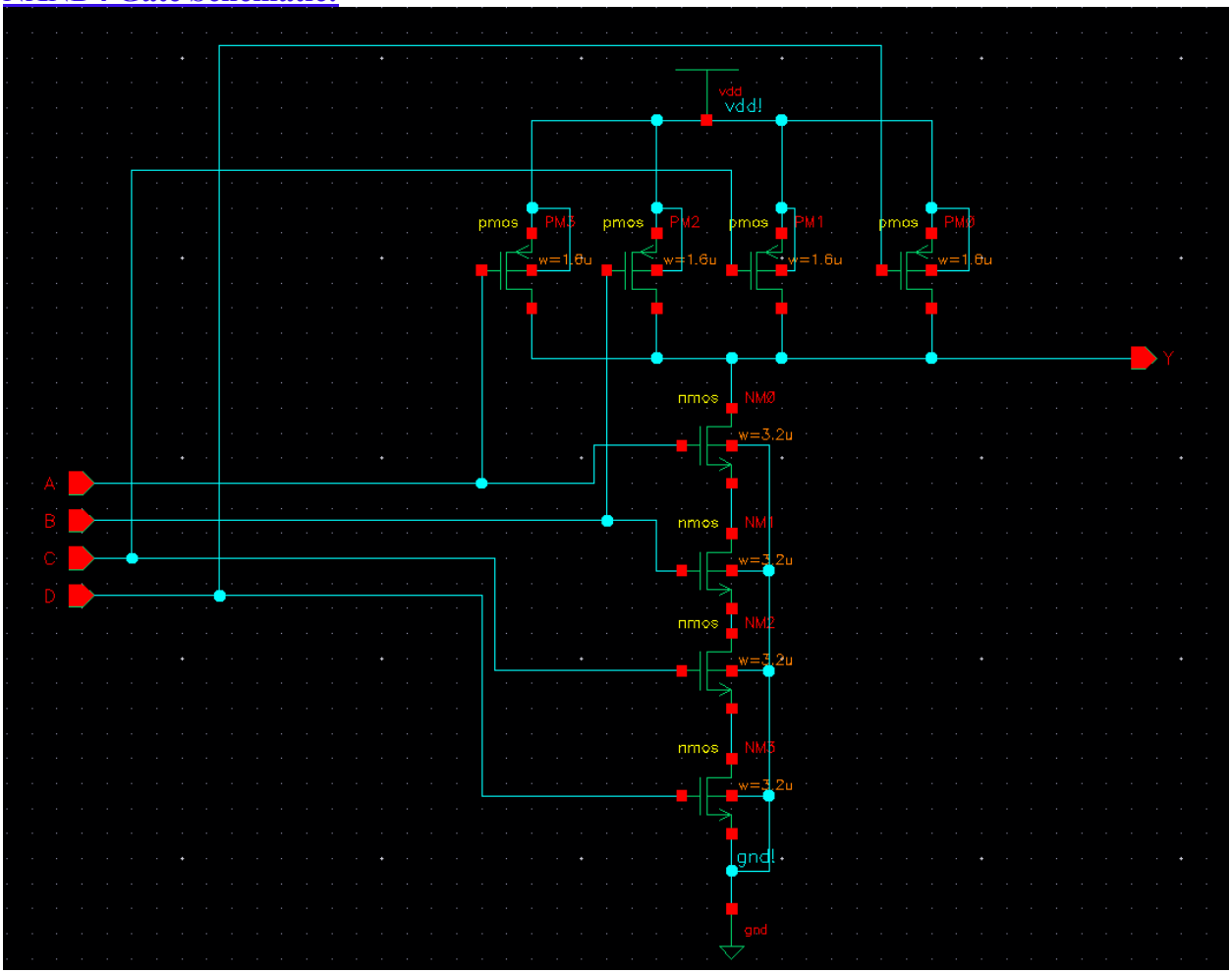


Figure 28-NAND4 Gate Schematic

NAND4 Gate Symbol:

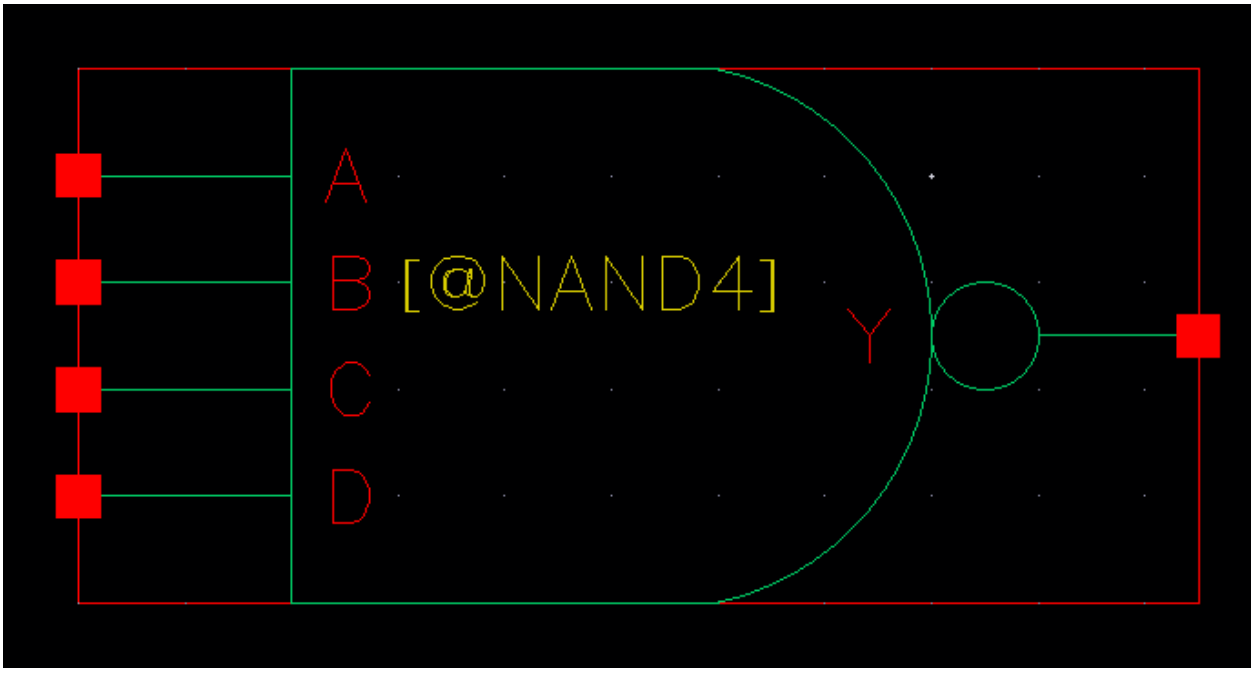


Figure 29-NAND4 Gate Symbol

NAND4 Gate Layout:

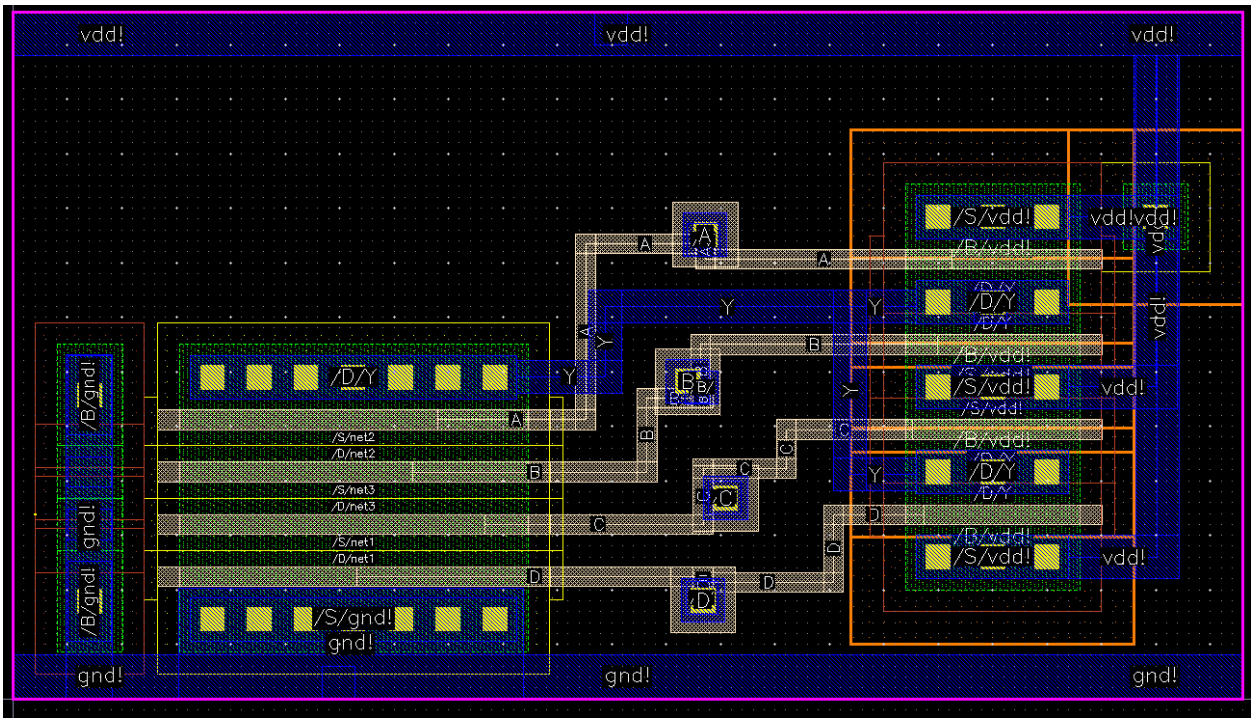


Figure 30-NAND4 Gate Layout

3 to 8 bit Decoder schematic:

First we build a decoder with minimum sized inverter .

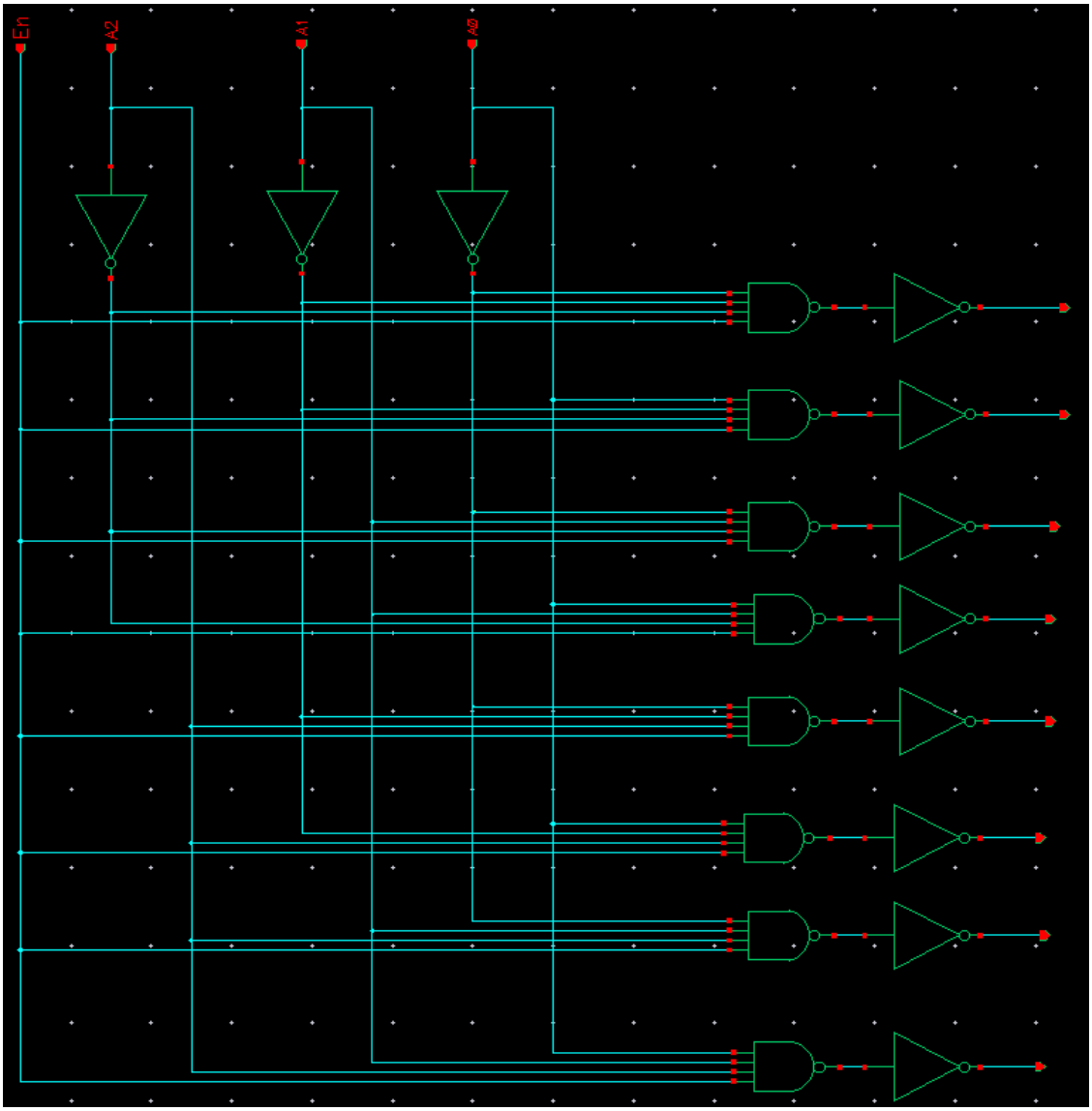


Figure 31 - 3 to 8 bit Decoder schematic

3 to 8 bit Decoder Symbol:

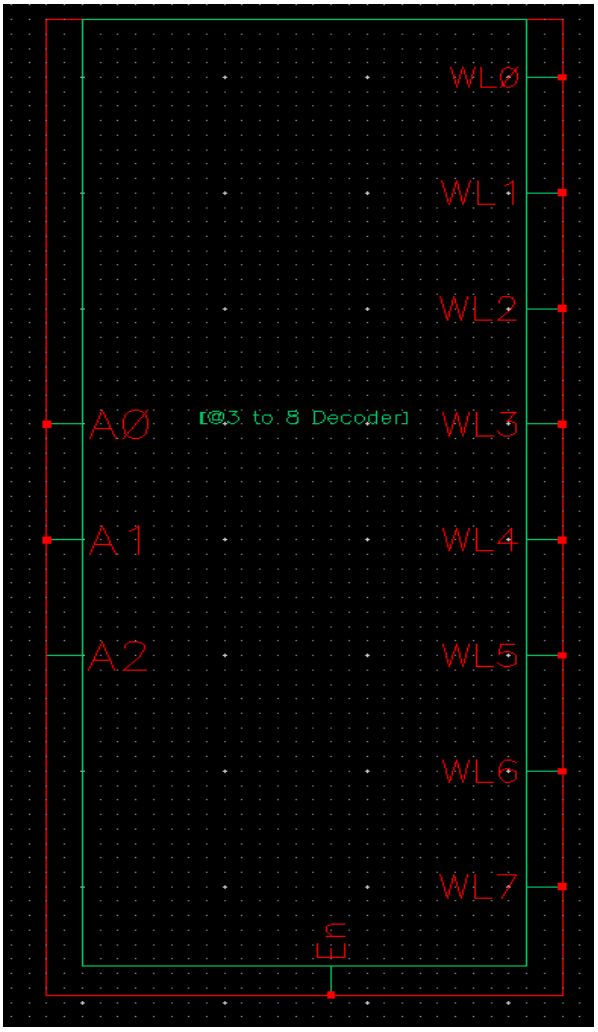


Figure 32 - 3 to 8 bit Decoder Symbol

3 to 8 Decoder Layout:

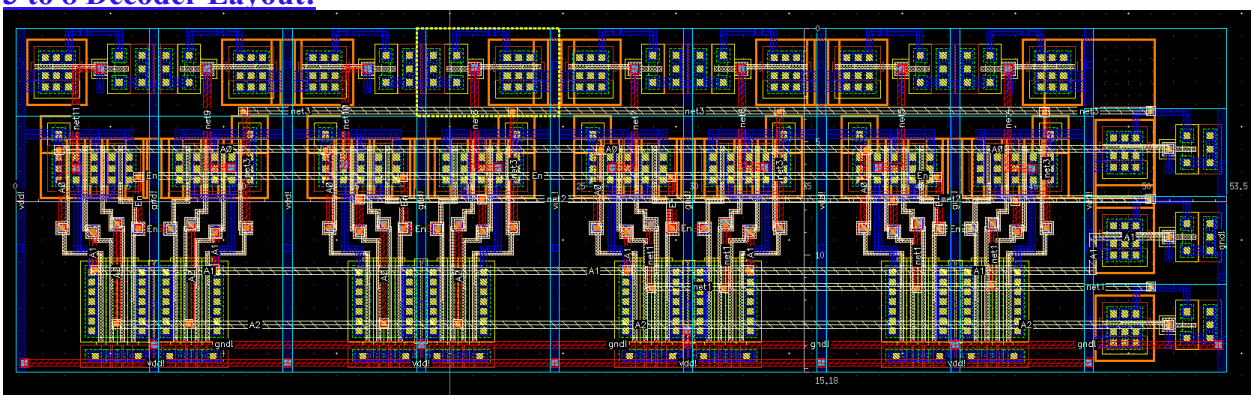


Figure 33-3 to 8 Decoder Layout

3 to 8 Decoder Stimulus:

```
simulator lang=spectre
global gnd!
vdd (vdd! 0) vsource dc=1.8
gnd (gnd! 0) vsource dc=0
V1 (A0 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=5n period=10n
V2 (A1 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=10n period=20n
V3 (A2 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=20n period=40n
V4 (En 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=55n period=80n
```

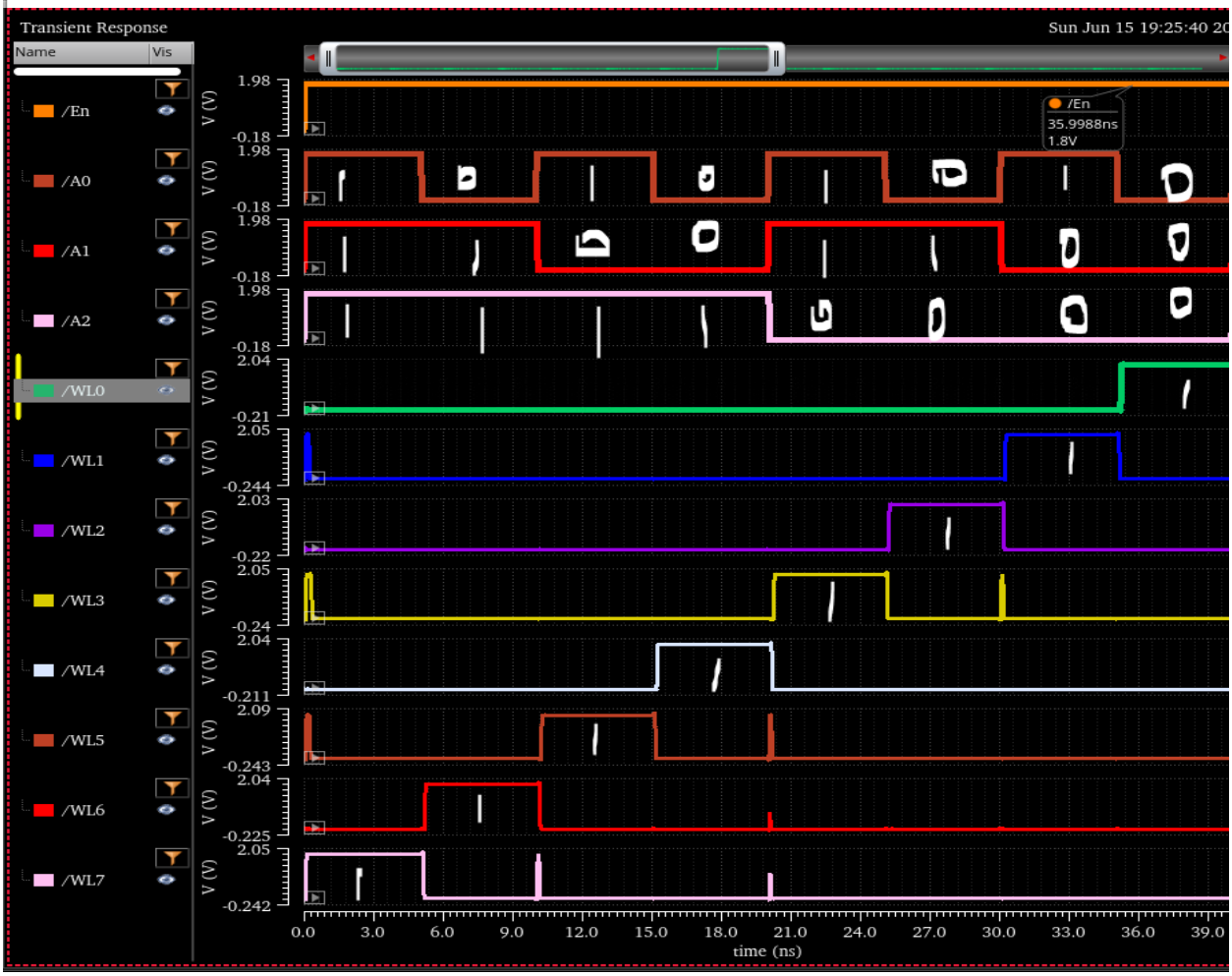


Figure 34 - 3 to 8 Decoder Waves

Decoder Delay Check:

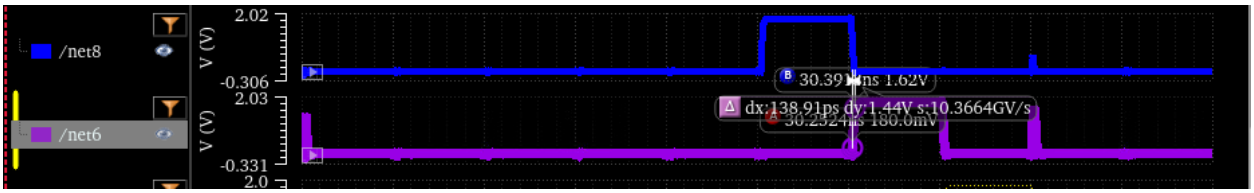


Figure 35-decoder rise time with minimum inverter size

After analyzing the timing performance of the decoder, we decided to improve the rise time by designing a larger inverter. Therefore, we implemented a double-sized inverter within the decoder to enhance its switching speed.

Bigger inverter schimatic :

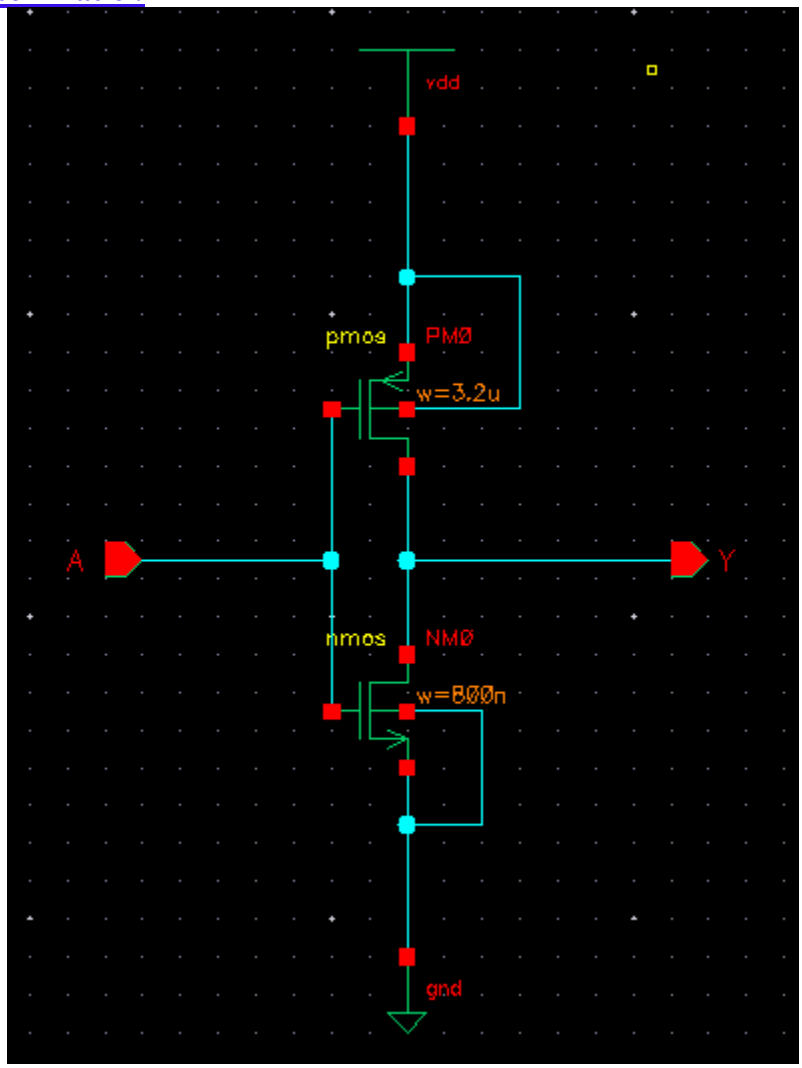


Figure 36-Bigger inverter schematic

Bigger inverter symbol :

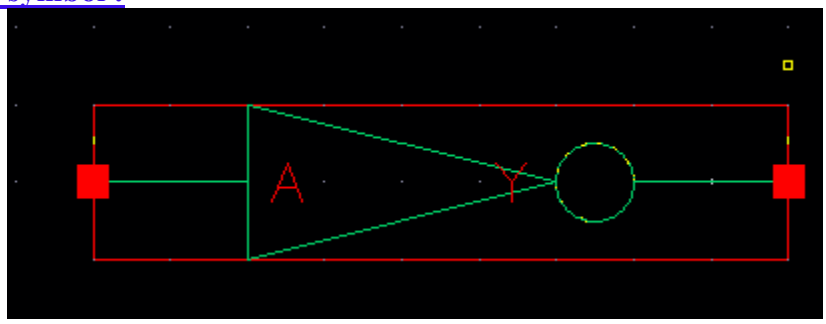


Figure 37-Bigger inverter symbol

Bigger inverter Layout:

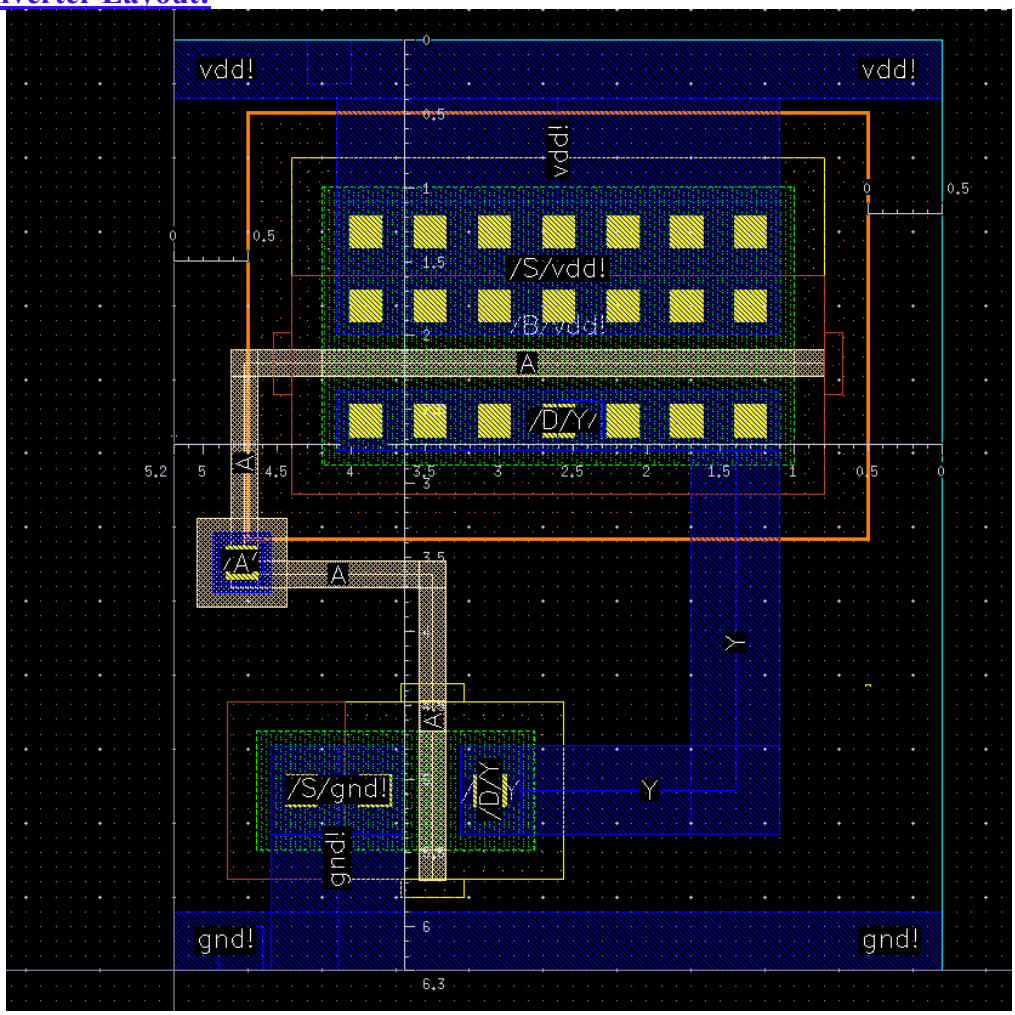


Figure 38-Bigger inverter Layout

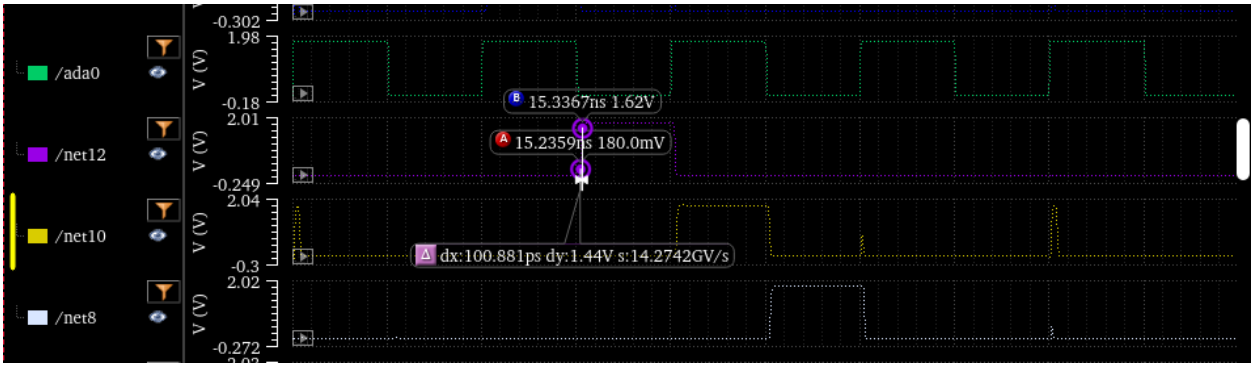


Figure 39-WL rise time big inverter

The simulation results show a noticeable improvement in rise time due to the implementation of the larger inverter.

3 to 8 Decoder Layout after inverter improvement:

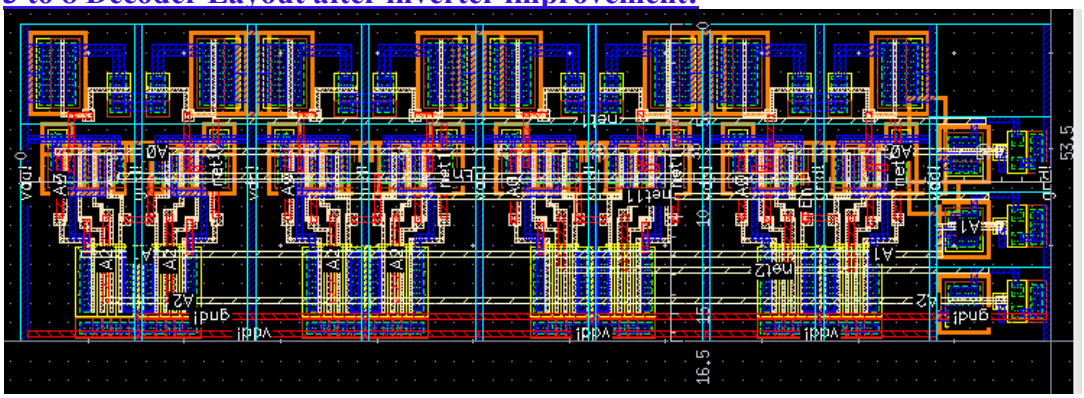


Figure 40-3 to 8 Decoder Layout after inverter improvement

SRAM integration:

After developing the individual building blocks (6T memory cell, RW access cell, and 3-to-8 decoders), we integrated them into a full 8×8 SRAM array. The array supports **read and write** operations. Each row is selected using a dedicated wordline driven by the decoder, with enable control. The memory cells are arranged in 8 rows and 8 columns, where each bitline is shared across all rows for a given bit. On the left side, the address inputs (ada<0:2>) and enable signal control the global read/write decoder and input output buffers.

The control signals rw and rfen manage the overall read/write functionality and enable behavior. To complete the system, we added a **sense amplifier** for fast and accurate readout, a **precharge circuit** to initialize the bitlines before each read, and **non-inverting tri-state buffers** on both the input and output paths for controlled data flow. This ensures that the outputs only drive the data bus when enabled, and remain in a high-impedance state otherwise.

Top Level SRAM

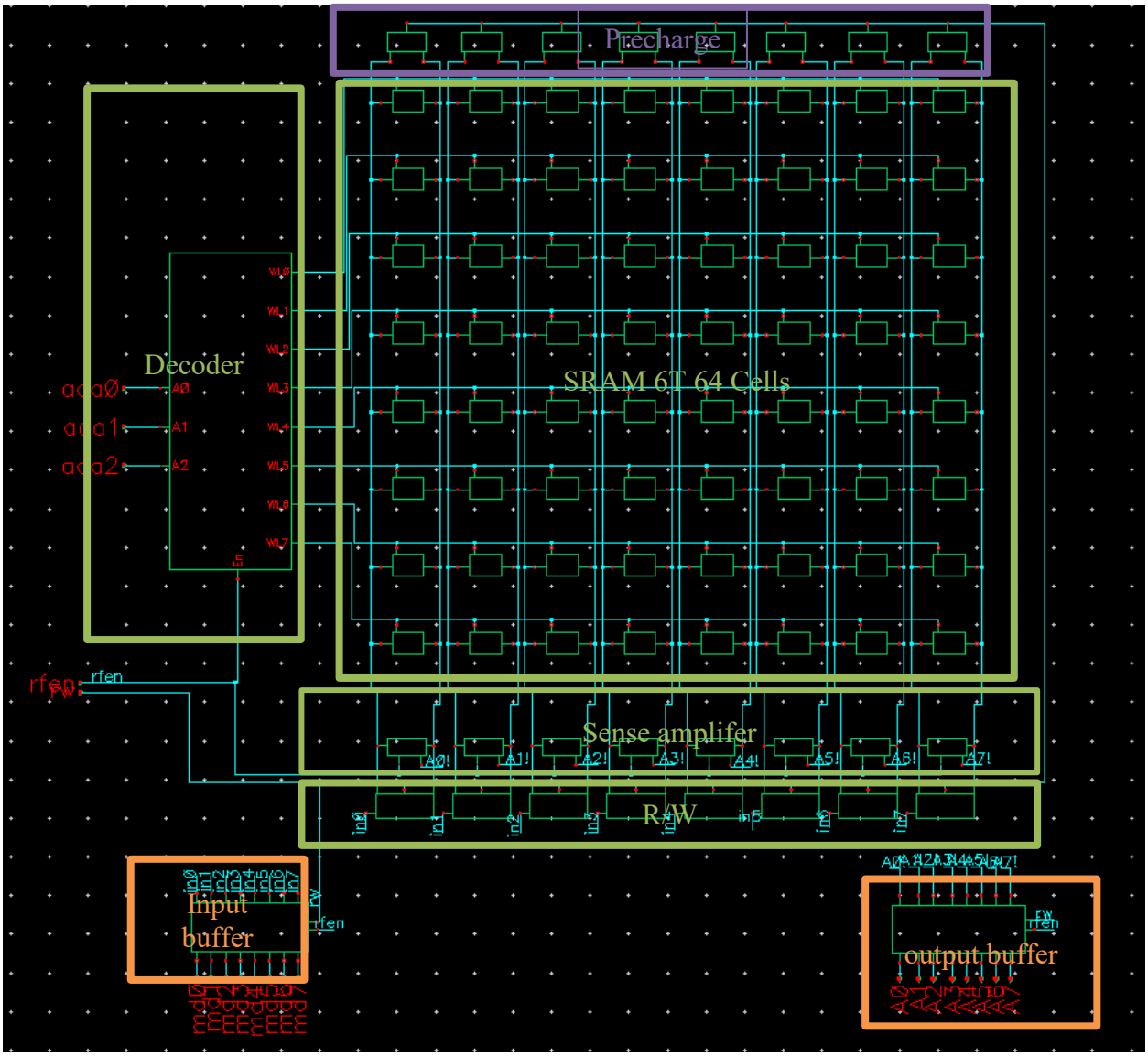


Figure 41-Top Level SRAM Schematic

SRAM Symbol:

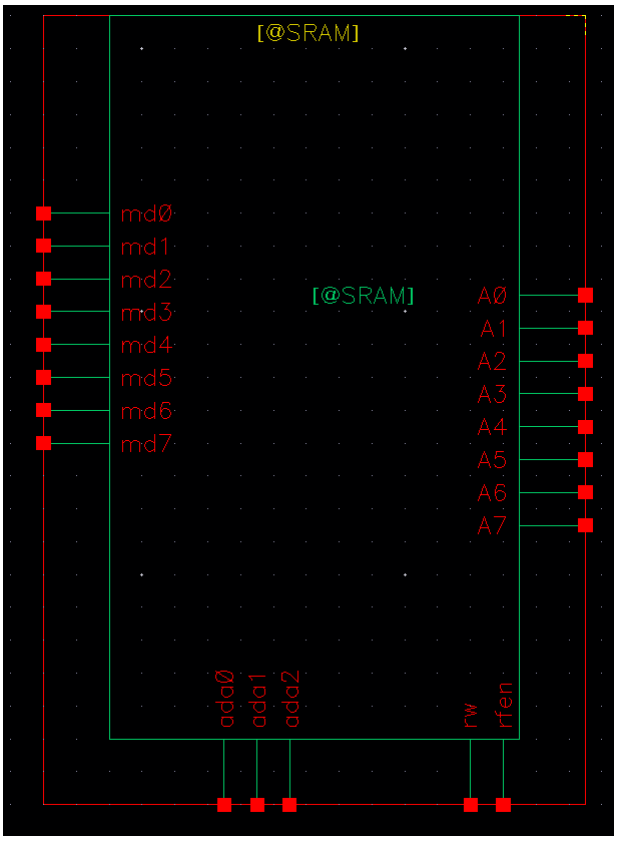


Figure 42- SRAM Symbol

SRAM Layout:

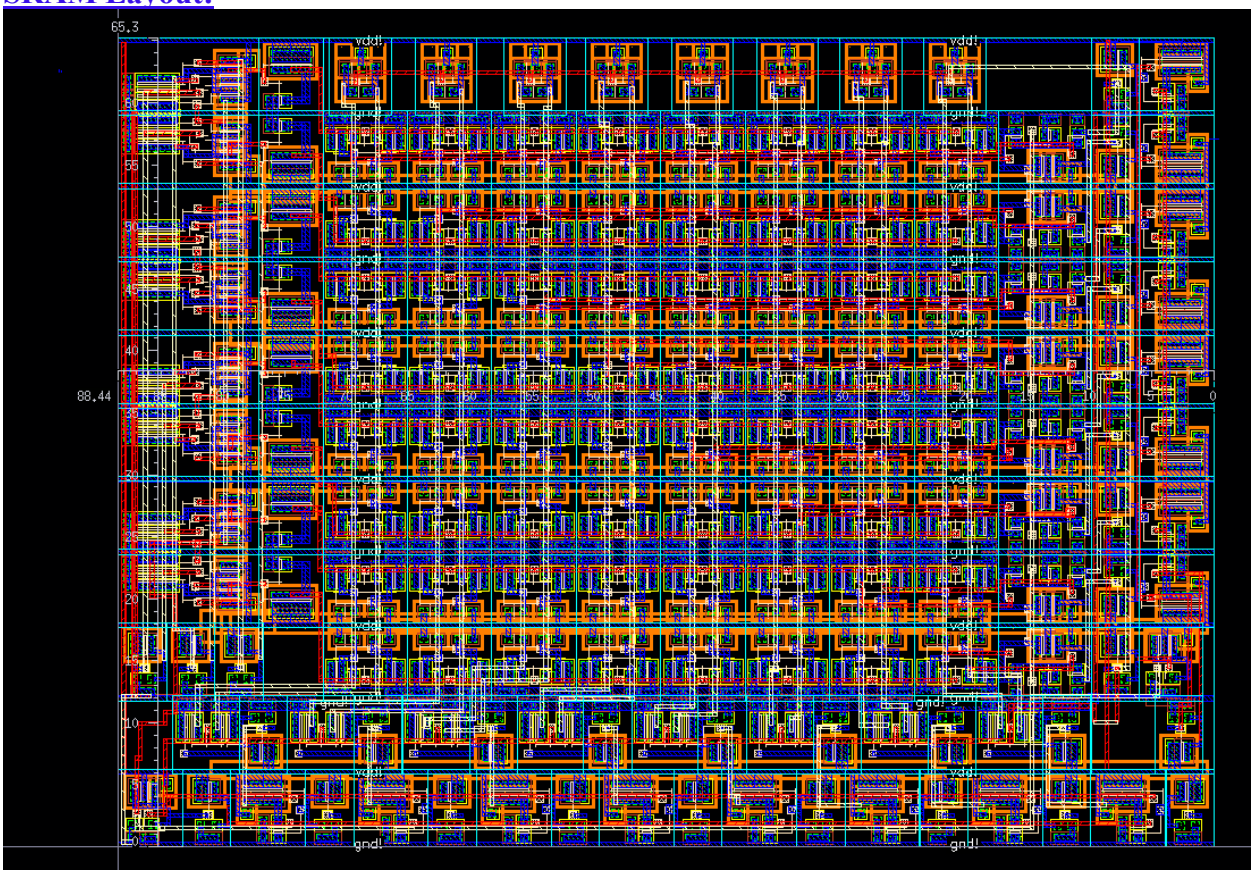


Figure 43-SRAM Layout

SRAM Testbench:

To verify the full SRAM array functionality, we created a detailed Spectre simulation environment with pulse and piecewise-linear (PWL) sources. The supply voltage was set to **1.8 V**, and logic '0' was tied to ground.

Data inputs (V0–V7) were defined using pulse sources with different periods and delays to simulate dynamic write values.

Address inputs for write and **read (V8–V10)** were generated using PWL waveforms that change values at specified nanosecond timestamps in this case we took a signal word address 010. this tests the WL2 world .

Enable signal (V14) was activated at 0.05 ns and held high for most of the simulation to see the full functional result.

Read/Write mode signal (V15) was set to toggle between `0` (write mode) and `1.8 V` (read mode) at defined intervals to alternate between writing and reading.

These input sources simulate realistic timing sequences and allow full control over cell addressing, data injection, and output behavior observation.

```
simulator lang=spectre
global gnd!
vdd (vdd! 0) vsource dc=1.8
gnd (gnd! 0) vsource dc=0
//data
V0 (md0 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=5n period=10n
V1 (md1 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=10n period=20n
V2 (md2 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=5n period=10n
V3 (md3 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=10n period=20n
V4 (md4 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=5n period=10n
V5 (md5 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=10n period=20n
V6 (md6 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=5n period=10n
V7 (md7 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=5n period=10n

//address for A global output input

V8 (ada0 0) vsource type=pwl wave=\[0n 0 50n 0]
V9 (ada1 0) vsource type=pwl wave=\[0n 0 0.05n 1.8 50n 1.8]
V10 (ada2 0) vsource type=pwl wave=\[0n 0 50n 0]

//address for B output

//V11 (adb0 0) vsource type=pwl wave=\[0n 0 50n 0]
//V12 (adb1 0) vsource type=pwl wave=\[0n 0 50n 0]
//V13 (adb2 0) vsource type=pwl wave=\[0n 0 50n 0]

//enable

V14 (rfen 0) vsource type=pwl wave=\[0n 0 0.05n 1.8 50n 1.8]

//read /write

V15 (rw 0) vsource type=pwl wave=\[0n 0 0.05n 1.8 25n 1.8 25.05n 0 50n 0]
```

Figure 44-SRAM Stimulus file

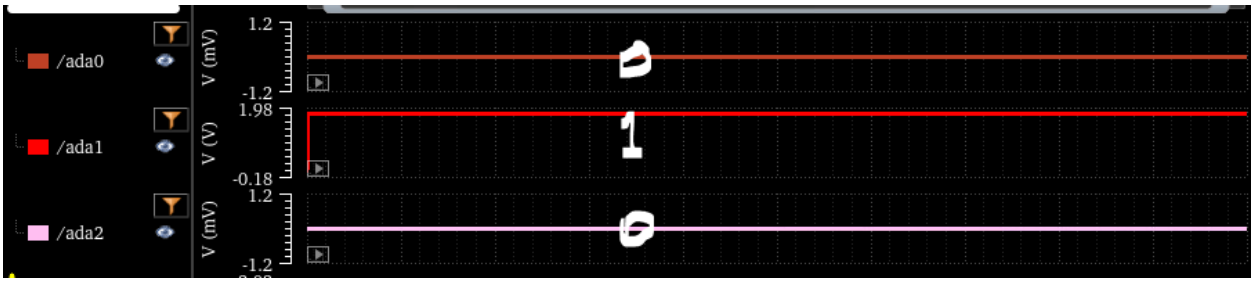


Figure 45- part 1 of SRAM simulation output decoder ADDRESS

The waveform shows the behavior of the SRAM array during controlled write cycles. When the rw signal is **high**, the input data bits md0–md7 are written into the memory cells. Each bitline reflects the expected logic value, indicating that data was correctly stored. When rw is **low**, no write occurs and the memory inputs remain at High impedance mode after the buffer .

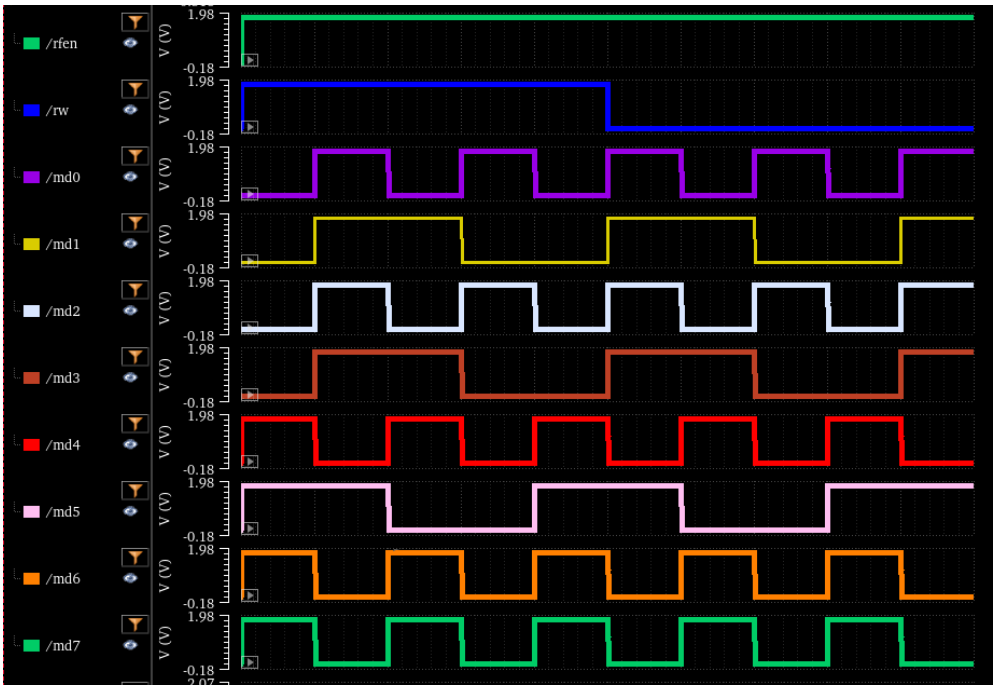


Figure 46 - part 2 SRAM read/write and Data inputs

The waveform below shows the behavior of the SRAM array during controlled write cycles. When the rw signal is **high**, the input data bits md0–md7 are written into the memory cells. Each bitline reflects the expected logic value, indicating that data was correctly stored. When rw is **low**, no write occurs and the memory inputs remain at High impedance mode after the buffer .

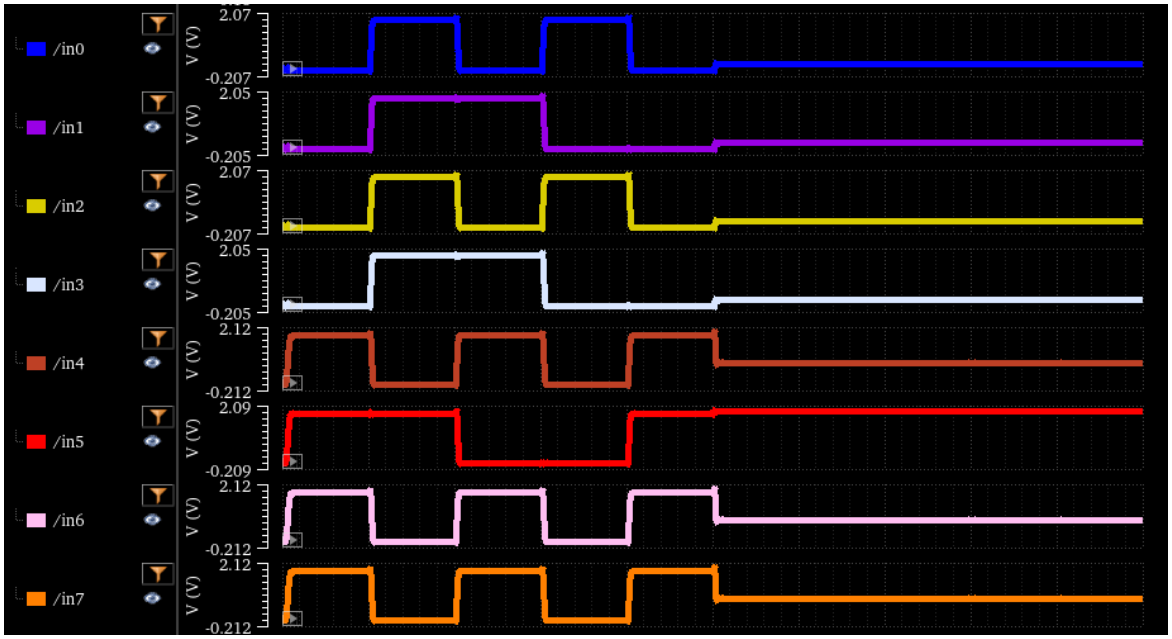


Figure 47- part 3 SRAM read/write Data inputs after input buffer

This waveform illustrates the output behavior of all eight SRAM bits (A0–A7). In this case the enable is on and When rw is **high(write cycle)**, the outputs from the sense amplifer track the input data values during the write phase; when rw goes **low**, the outputs hold and reflect the final stored values from the selected memory row, confirming successful data retention and readback.

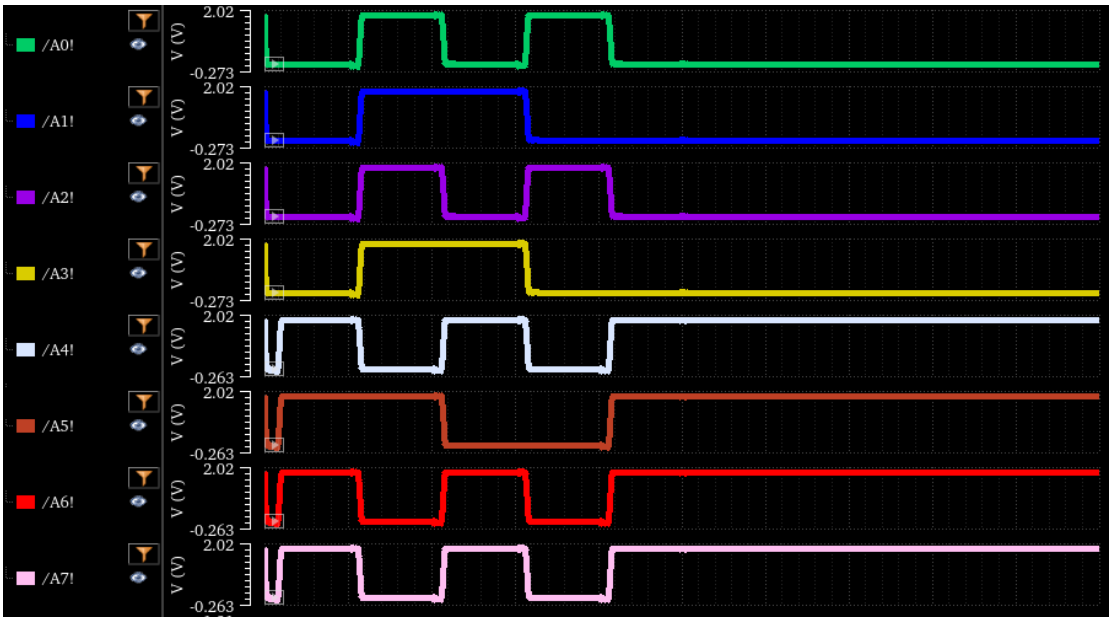


Figure 48-part 3 SRAM Data outputs after sense amplifer

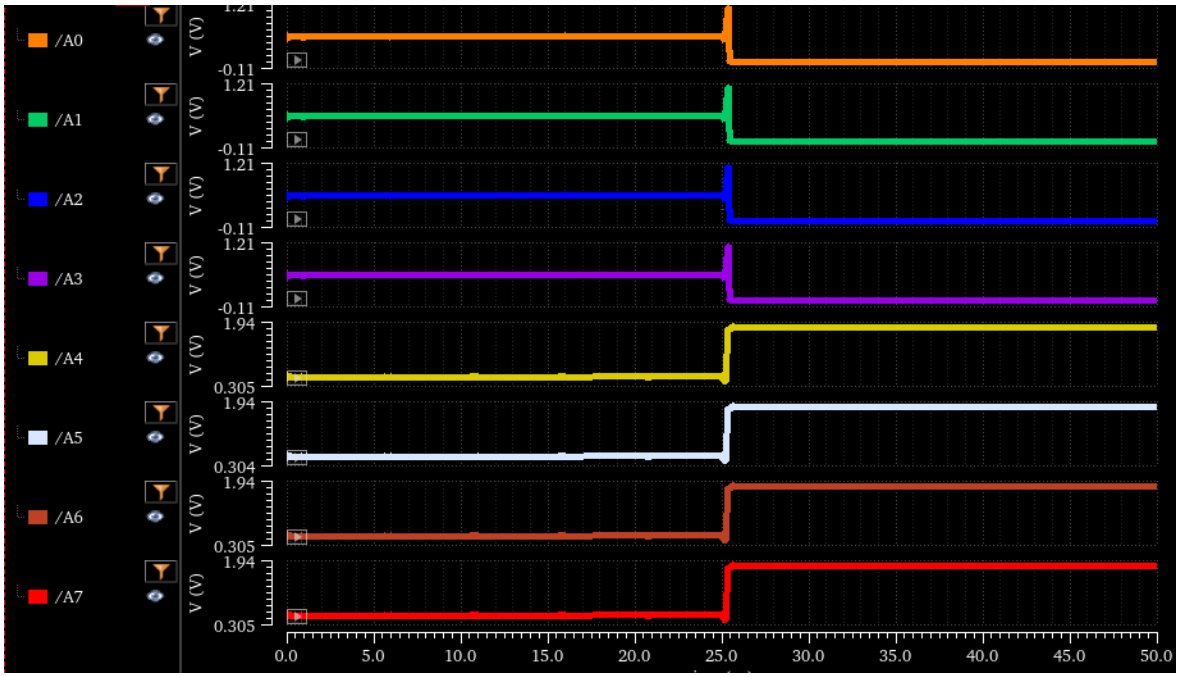


Figure 49-part 3 SRAM read/write Data outputs after buffer

This waveform illustrates the final output behavior of the SRAM system **after the tri-state output buffers**. When enable is low (write cycle), the outputs enter a **high-impedance (High-Z)** state, effectively disconnecting from the bus and preventing data contention. This behavior confirms that the tri-state buffers successfully isolate the SRAM outputs unless explicitly enabled, ensuring safe and controlled data access.
When the enable signal is active, the buffered outputs (A0–A7) follow the values produced by the sense amplifier, allowing the data to propagate to the external bus.

We made the same stimulus for the other SRAM words and they give the same result .

Latch:

In our SRAM datapath , the latch serves as a temporary storage element to hold data during controlled timing phases. It is particularly used to capture and hold input or output data during clk1 cycle. When the clock signal is high, the latch is transparent and allows data to pass through. When the clock goes low, the latch holds the last valid value, isolating the memory cell from unwanted changes. This ensures reliable synchronization between memory access and control signals without data corruption.

Latch schematic:

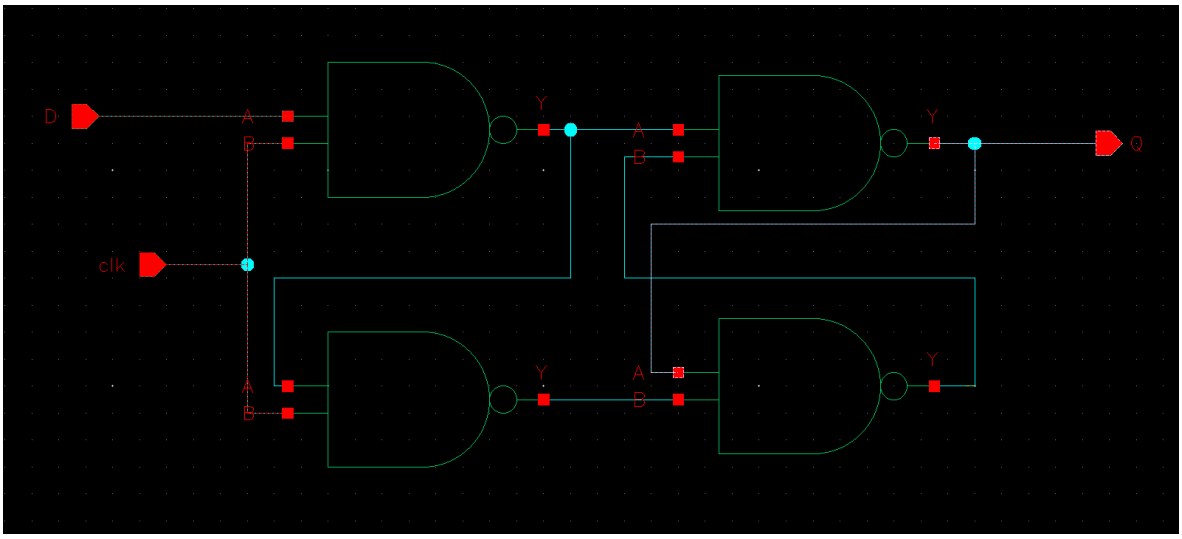


Figure 50- Latch schematic

Latch Symbol:

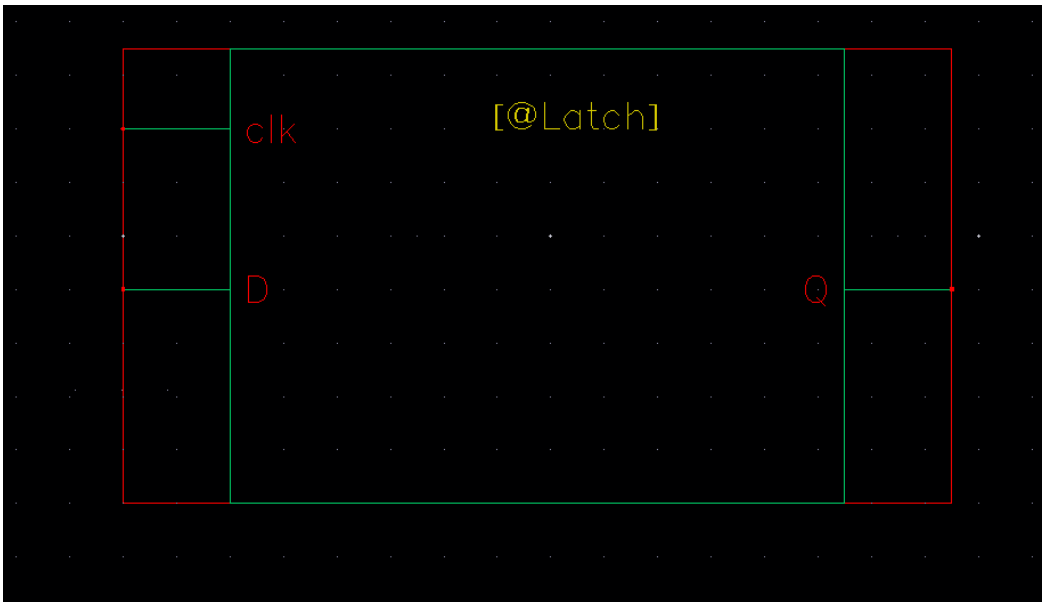


Figure 51-Latch Symbol

Latch Layout:

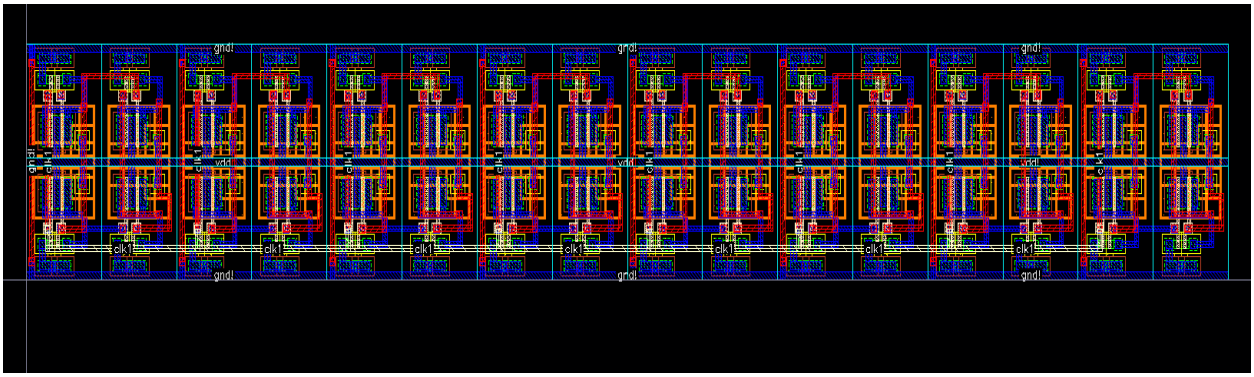


Figure 52-Latch Layout

1 bit Latch Stimulus:

```
simulator lang=spectre
global gnd!
vdd (vdd! 0) vsource dc=1.8
gnd (gnd! 0) vsource dc=0
V1 (clk 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=5n period=10n
V4 (D 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=10n period=20n
```

Figure 53-Latch Stimulus

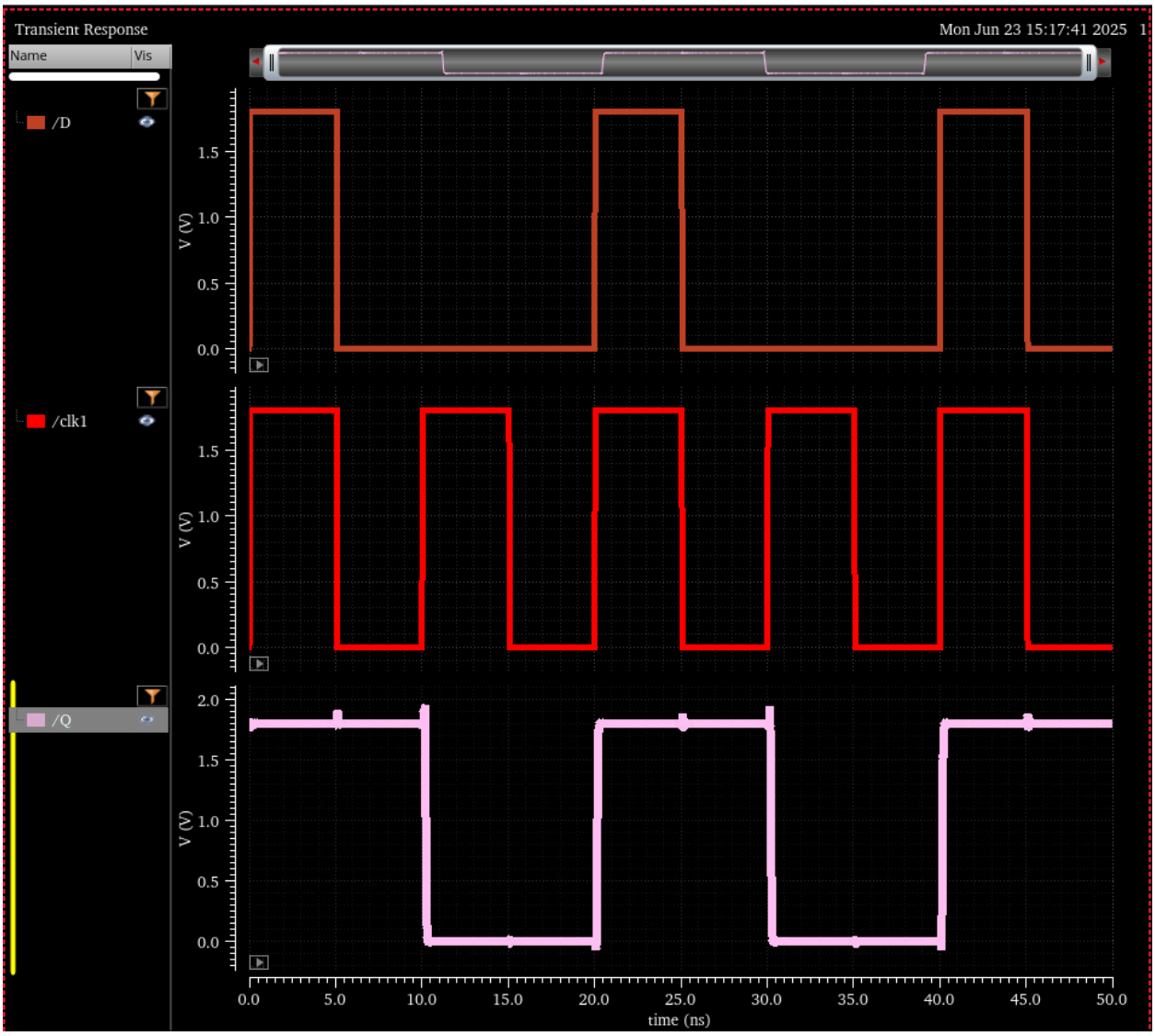


Figure 54-Latch Stimulus result

Final Integration:

Final project integration latch with SRAM :

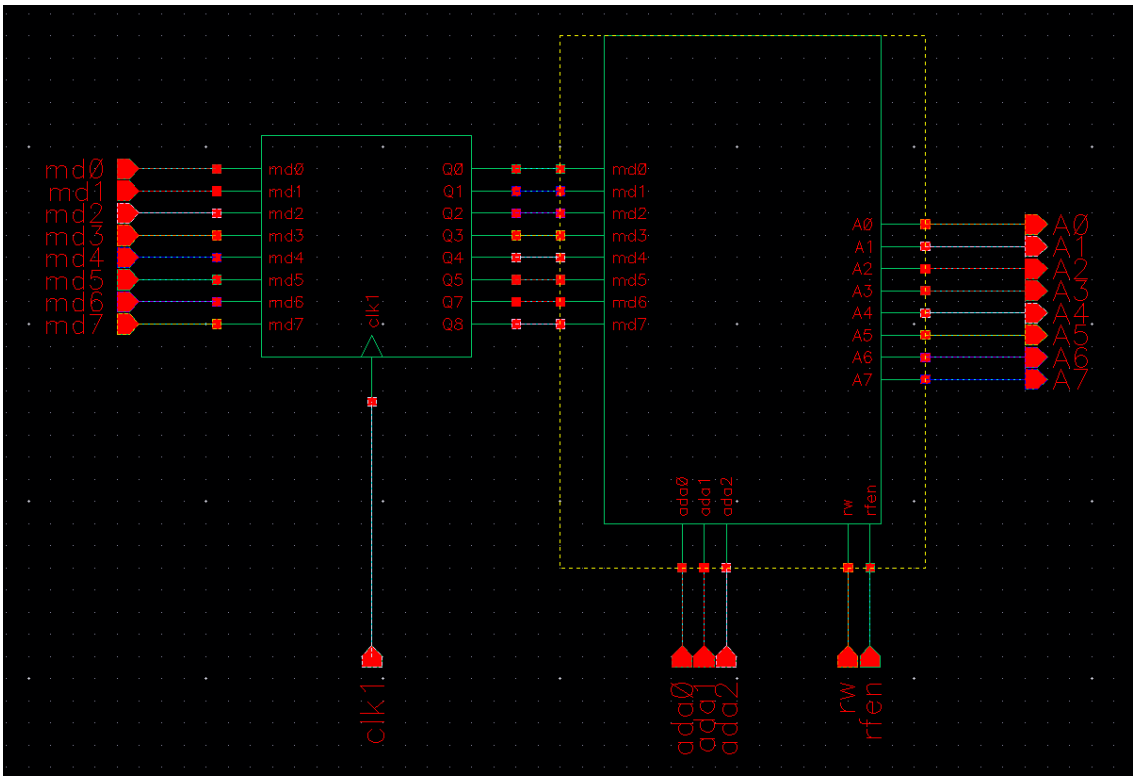


Figure 55-Final project integration Latch with SRAM

Final Integration Layout:

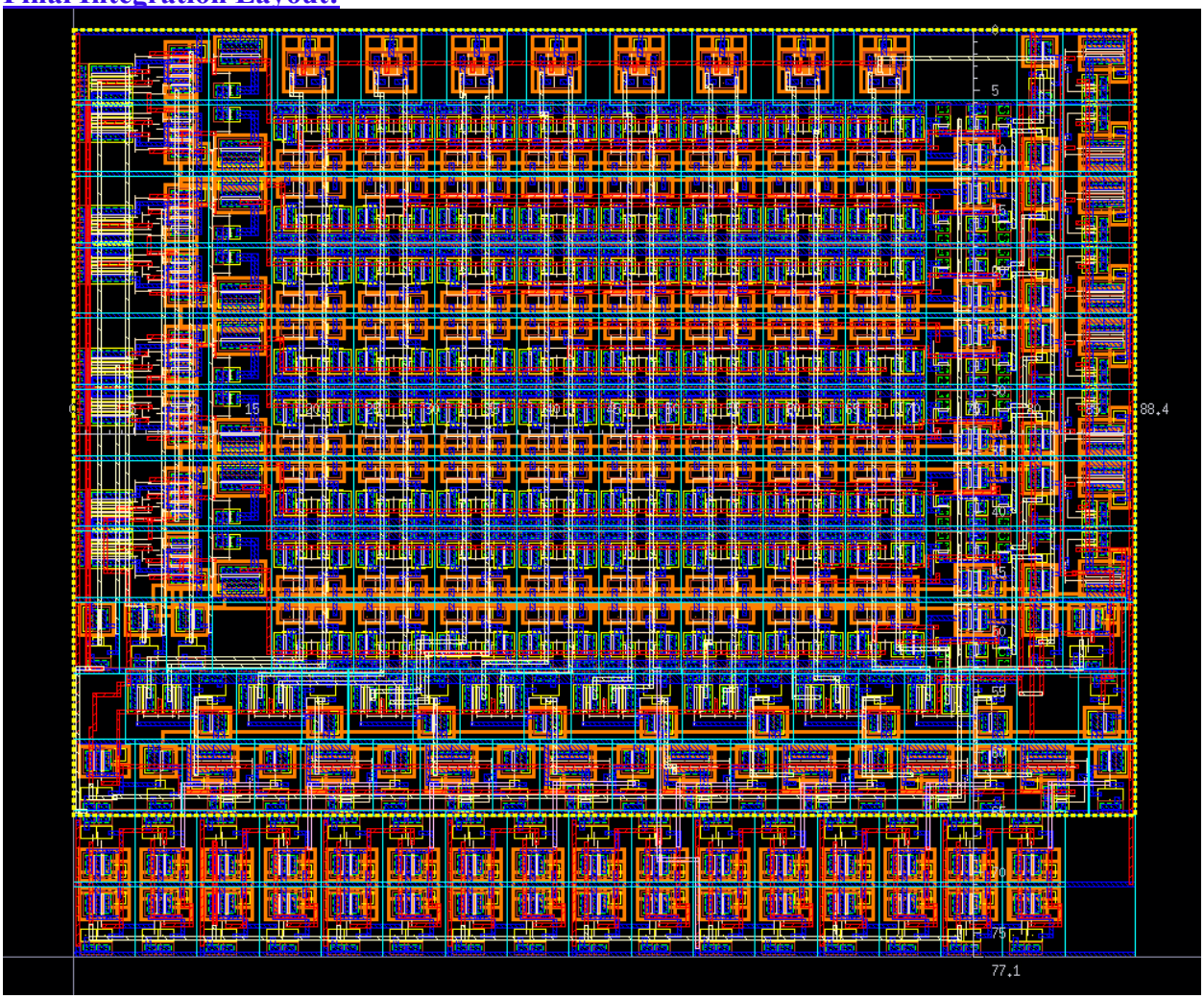


Figure 56-Final Integration Layout SRAM +8bit Latch

Final integration testbench:

Open *SRAM_Testbench+Latch.txt ~/stimulus_texts/final_proj Save

```
simulator lang=spectre
global gnd!
vdd (vdd! 0) vsource dc=1.8
gnd (gnd! 0) vsource dc=0

//data

V0 (md0 0) vsource type=pwl wave=\[0n 0 2n 0 2.05n 0 5n 0 20n 0 20.05n 1.8 50n 1.8]
V1 (md1 0) vsource type=pwl wave=\[0n 0 2n 0 2.05n 1.8 5n 1.8 5.05n 0 50n 0]
V2 (md2 0) vsource type=pwl wave=\[0n 0 2n 0 2.05n 1.8 5n 1.8 5.05n 0 50n 0]
V3 (md3 0) vsource type=pwl wave=\[0n 0 2n 0 2.05n 1.8 5n 1.8 5.05n 0 50n 0]
V4 (md4 0) vsource type=pulse val0=0 val1=1.8 delay=5n rise=0.05n fall=0.05n width=5n period=10n
V5 (md5 0) vsource type=pulse val0=0 val1=1.8 delay=5n rise=0.05n fall=0.05n width=5n period=10n
V6 (md6 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=5n period=10n
V7 (md7 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=5n period=10n

//address for A global output input

V8 (ada0 0) vsource type=pwl wave = \[0n 0 60n 0]
V9 (ada1 0) vsource type=pwl wave = \[0n 0 0.05n 1.8 60n 1.8]
V10 (ada2 0) vsource type=pwl wave = \[0n 0 60n 0]

//enable

//V14 (rfen 0) vsource type=pwl wave=\[0n 0 0.05n 1.8 50n 1.8]

V14 (rfen 0) vsource type=pulse val0=0 val1=1.8 delay=7n rise=0.05n fall=0.05n width=3n period=10n

//read /write

//V15 (rw 0) vsource type=pwl wave=\[0n 0 0.05n 0 25n 0 25.05n 1.8 50n 1.8]

V15 (rw 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=10n period=20n

//clk1

V16 (clk1 0) vsource type=pulse val0=0 val1=1.8 delay=2n rise=0.05n fall=0.05n width=3n period=10n
```

Figure 57-Final Integration stimulus

We built the complete datapath circuit according to the project specifications, integrating the register file, latches, decoders, and control signals. A stimulus file was written in Spectre using timing patterns aligned with the waveform described in the project Description. This included synchronized control of `clk1`, `rfen`, `rw`, address signals, and input data lines (`md0-md7`). The waveform generator enabled proper sequencing of read and write cycles, ensuring that latches open and close at the right clock phases and that the SRAM array responds accordingly. The setup allowed us to simulate the full 2-phase datapath: reading data from the register file to the ALU and writing results back.

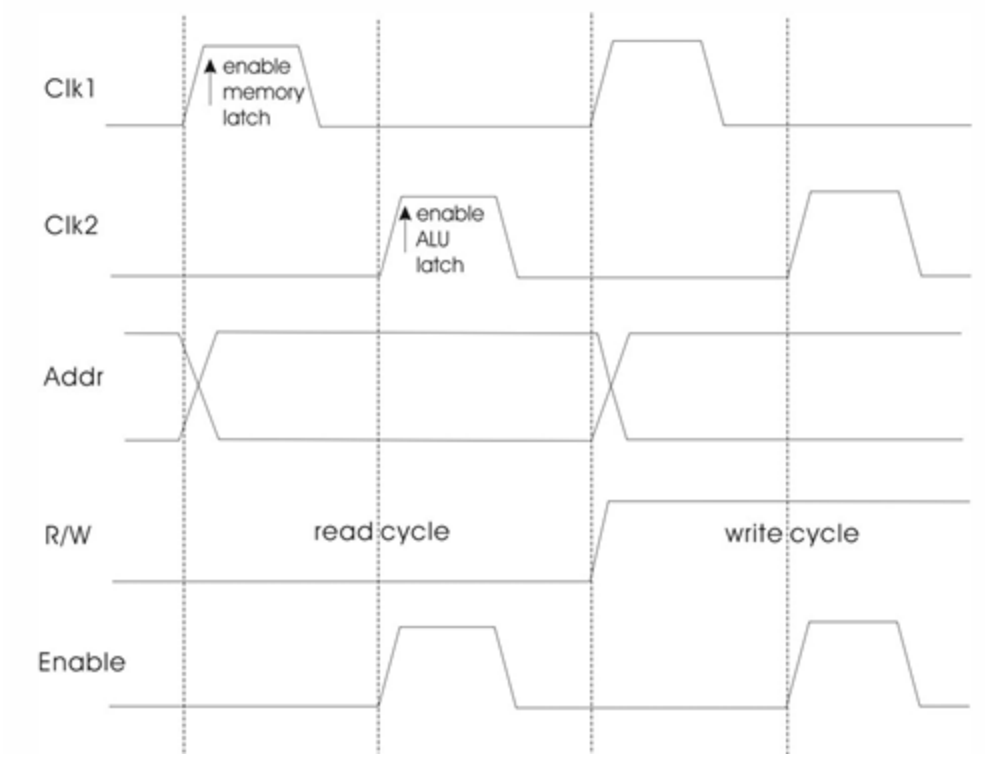


Figure 58-Timing diagram for a complete 2-cycle data path function from project description.

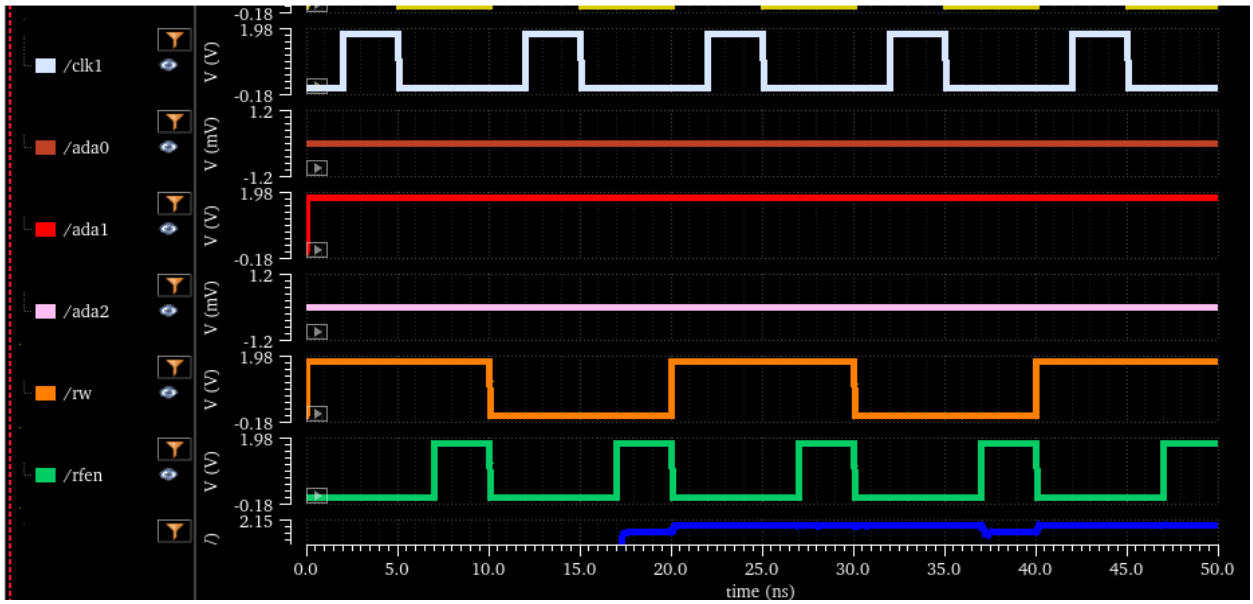


Figure 59-Timing diagram for data path function wave

Final result for one cell :

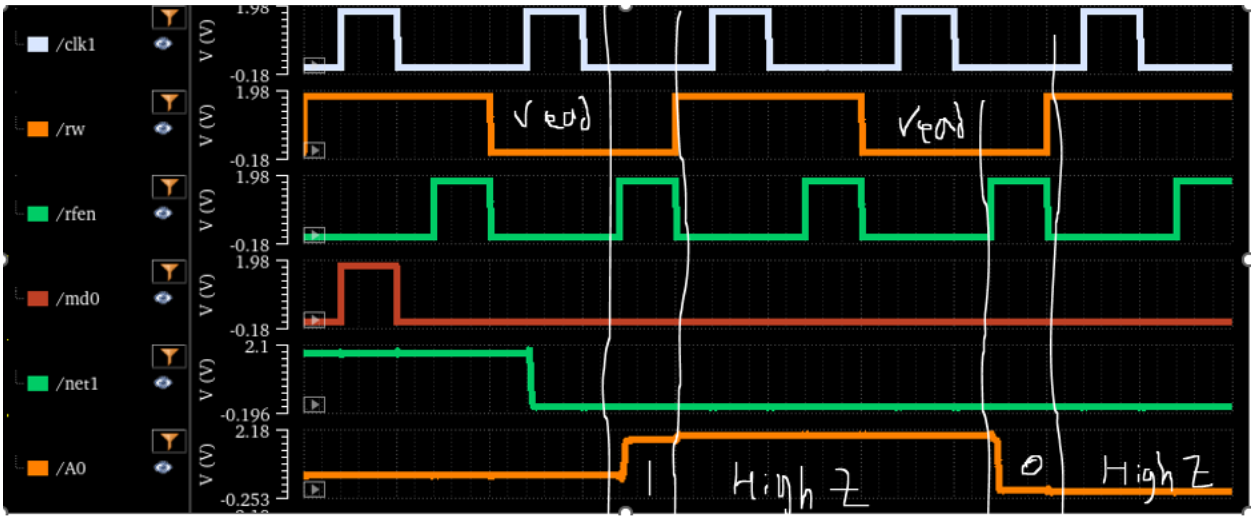


Figure 60- summary result for one cell

The waveform demonstrates the correct behavior of the SRAM datapath with respect to the latch and enable control. When clk1 is high, the input data [md0] is captured by the latch (net1) and held as long as clk1 is low. During the write cycle when [rw] high and [rfen] high , the SRAM stores the latched value. In the following read cycle [rw] low and [rfen] high, the previously written value is correctly output on [A0](after the enable the output entered to High Z mode). In the second [clk1] pulse, the latch captures a LOW input value, which remains stable throughout the rest of the simulation. This low value is stored during the next write cycle and correctly appears at the output during the final read cycle, confirming full synchronization between the latch, SRAM write, and tri-state output logic.](after the enable the output entered to High Z mode)

Timing:

After further optimization and analysis, the updated worst-case propagation delay was measured as **402.46 [ps]**, allowing the SRAM array to theoretically operate at up to **2.49 GHz**. This speed confirms that the design is highly efficient and well within the project's 5 ns delay requirement.

$$f_{Max} = \frac{1}{T_{worstcase}} = \frac{1}{402.46[PS]} = 2.49\ [GHz]$$

write access delay:

The **write access delay** is defined as the time from the activation of clk1 and rw until the data is successfully stored in the cell.

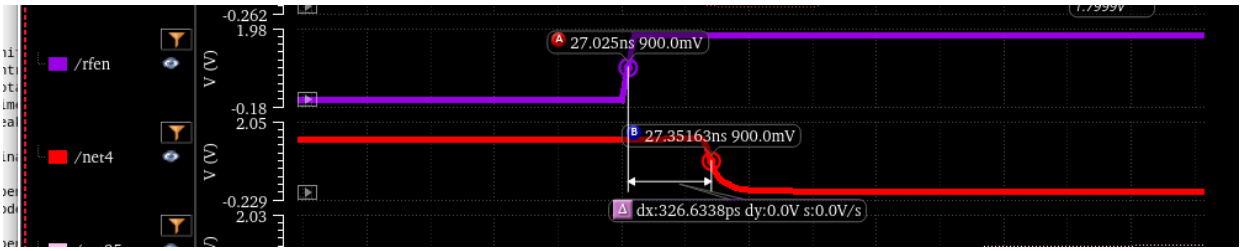


Figure 61-Write Access Propagation Delay High to Low & Low to High

The write delay is similar for both high-to-low and low-to-high transitions because writing is performed by pulling the inverted node of the 6T cell to 0, allowing the cell to flip its state through the internal cross-coupled inverters .

read access delay:

The **read access delay** measures the time from the activation of the enable signal to the moment the correct data appears at the output after passing through the sense amplifier and tri-state

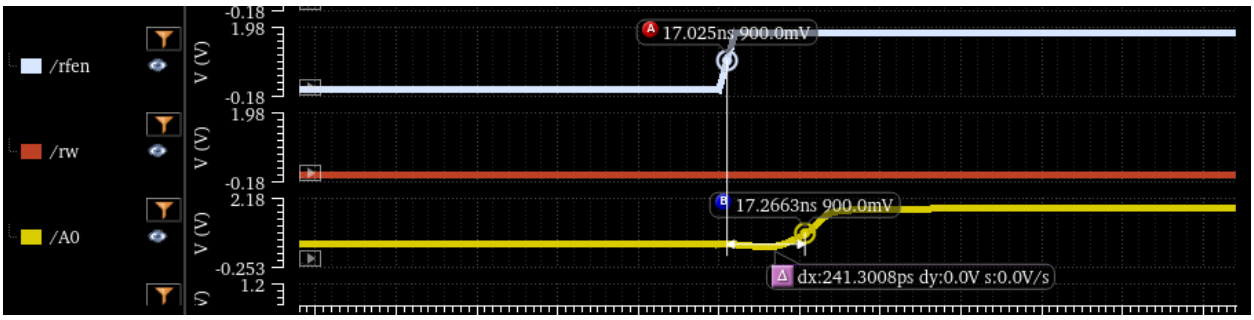


Figure 62-Read Access Low to High Propagation Delay

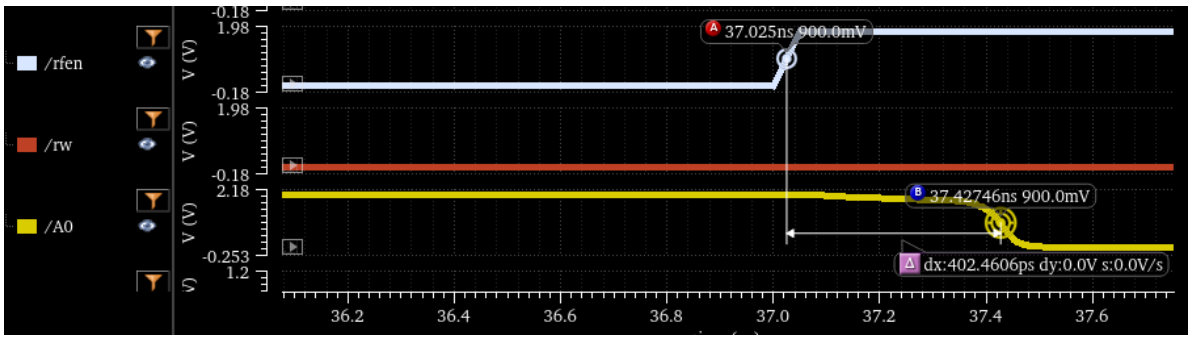


Figure 63 - Read Access High to Low Propagation Delay

bitline discharge time:

The bitline discharge time reflects how quickly the bitline drops from VDD, affecting the sense amplifier's ability to detect a valid logic level.

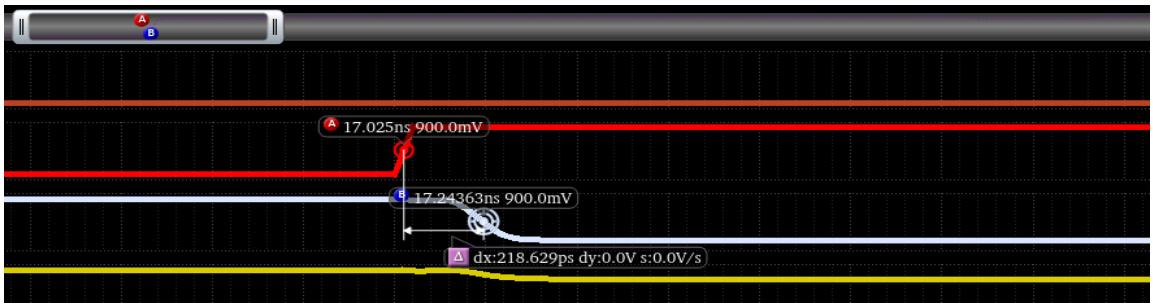


Figure 64-bitline discharge time

The worst case **enable-to-output delay** captures the latency between enabling the tri-state buffer and seeing the output data.
Is Read Access Propagation Delay Low to High with dt= 402.46[ps].

Output Rise and Fall time:

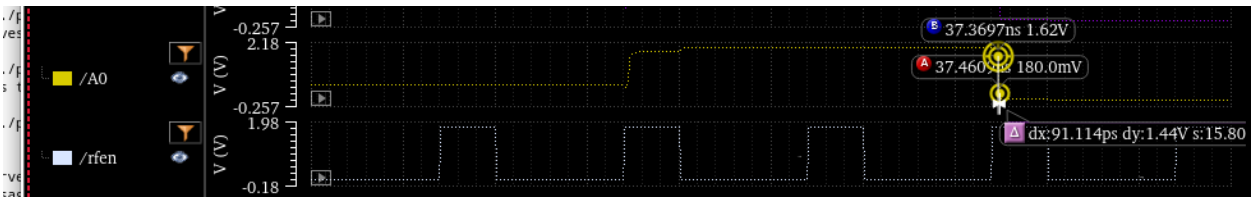


Figure 65-output fall time

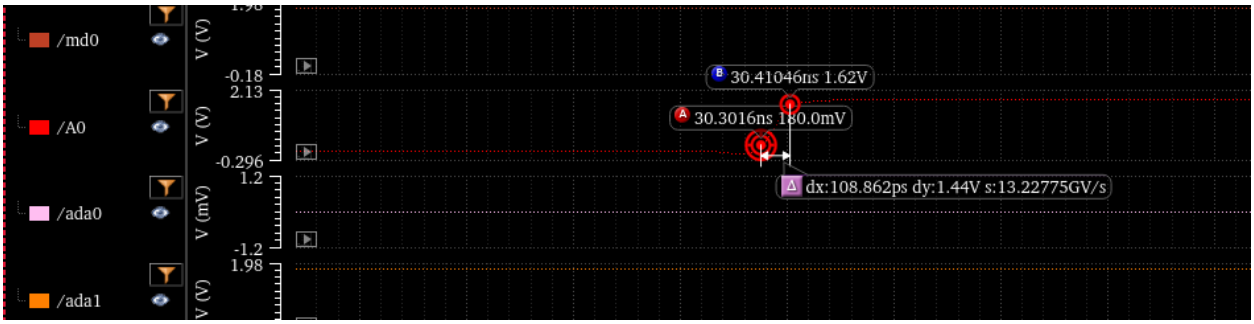


Figure 66-output rise time

	A Low to High	A High to Low
Read Access Propagation Delay	241.3 [ps]	402.46[ps]
Write Access Propagation Delay	326.633 [ps]	326.633[ps]
bitline discharge time	218.629[ps]	
Output Rise time	108.862[ps]	
Output Fall time	91.11[ps]	

Table 2-Timing table.

Conclusions:

Layout Area:

The SRAM Layout has an area of 88.4 *65.3 = 5772.52 (um^2).

The fall system Layout SRAM +Latch ➔ 88.4*77.1=6815.64(um^2).

Metal Layers:

In our SRAM layout design, we utilized Metal 2 and Metal 3 layers strategically to optimize routing efficiency and minimize interconnect length. These higher metal layers were used primarily for:
Reducing wire length by enabling more direct routing paths across the array,

Avoiding congestion in lower layers (e.g., Metal 1) where transistor connections are dense,

Connecting different functional blocks of the SRAM, such as decoders, sense amplifiers, and memory cells, across rows and columns.

Number of Transistors:

The total number of transistors in the design calculated as follows:

1. **Memory Array:**
   - 64 cells (8×8), each with 6 transistors → **64 × 6 = 384 transistors**
2. **Read/Write Access Cells:**
   - 6 transistors per access cell × 8 → **6 × 8 = 48 transistors**
3. **Decoder:**
   - 3-to-8 decoder : 8 transistors per NAND4 cell and 2 per inverter
   - Assuming 8 bitlines : (8+2)*8=80 **transistors**
   - 3 inverters to address inputs 3*2 =6
   - 86 **transistors per decoder**
4. **Sense Amplifiers & Precharge:**
   - 5 transistors per sense amplifier and 3per Precharge
   - Assuming 8 bitlines →**(5+3)*8 transistors→ 64 transistors**
5. **Tri-State Buffers:**
   - Each uses 6 transistors
   - For 8 data bus → 48**transistors**
   - **2 NAND2 and 3 inverters to enable and rw control signals**
   - **input and output →96+14=110 transistors**
6. **Latch: 4 NAND2 per bit** Assuming 8 bitlines **→ (4*4)*8= 128 transistors**

Number of the transistors per SRAM Top Level: **692 transistors**
Number of the transistors per project Top Level SRAM+Latch:678+128=**820 transistors**

Conclusion table :

Cell Name	Cell width	Cell height	Num of transistors
6T_Cell	6.82um	6.3um	6
R/W circuit	9.5um	6.3um	6
Precharge	4.98um	6.3um	3
Sense Amplifer	7.46um	6.3um	5
Inverting tri-state	6.04um	6.3um	4
8 bit output buffer	9.92 um	59.4um	56
8 bit input buffer	88.44 um	6.3um	54
NAND4	11.3 um	6.3um	8
3 to 8 bit Decoder	53.5 um	16.5um	86
SRAM	88.44 um	65.3um	692
Latch 8 bit	82.56 um	12.2um	128
SRAM +Latch	88.4 um	77.1um	820

Table 3-size conclusion table

Cell Name	Worst case propagation delay	Output Rise time	Output Fall time
R/W circuit	30[ps]	X	90[ps]
Precharge	339.5[ps]	261[ps]	X
Inverting tri-state	75[ps]	96[ps]	91[ps]
8 bit output buffer	134[ps]	108[ps]	91.11[ps]
8 bit input buffer	134[ps]	108[ps]	91[ps]
NAND4	164[ps]	87.65[ps]	96.91[ps]
3 to 8 bit Decoder	194[ps]	35[ps]	30.2445[ps]
SRAM	402.46[ps]	108.862[ps]	91.11[ps]

Table 4-timing conclusion table

Summary of Experience:

This project provided valuable hands-on experience in full-custom VLSI design, from initial transistor-level schematics to layout, simulation, and system-level integration. We began by analyzing the theoretical requirements for stable SRAM cell operation and applied these principles in the design of a 6T memory cell. From there, we expanded the system to include all essential components: precharge circuits, sense amplifiers, tri-state buffers, decoders, and latches.

Throughout the project, we deepened our understanding of critical VLSI design concepts such as transistor sizing for performance and stability, bitline discharge behavior, and timing synchronization across a datapath. We also gained practical skills in using the Cadence Virtuoso design environment, including schematic capture, layout editing, DRC/LVS verification, and Spectre simulation.

Simulating and debugging each module—especially under dynamic conditions—helped us appreciate the importance of testbenches and proper waveform design. Building the complete SRAM datapath, including dual clock phases and High-Z control logic, allowed us to observe real-world timing delays and performance bottlenecks.

Working collaboratively, we managed to overcome several design challenges related to layout constraints, signal integrity, and integration. The final result—a fully functional, high-speed 8×8 SRAM with latches and buffers—demonstrates a strong grasp of VLSI memory design principles and the ability to bring a complex digital subsystem from concept to silicon-ready layout.

Time Required:

Estimated time spent on this lab assignment: 100 hours.

Feedback:

This project was both challenging and rewarding, providing a comprehensive learning experience in digital VLSI design. It required us to apply theoretical knowledge in a practical context, transitioning from basic transistor-level understanding to full system integration.

One of the most valuable aspects was the opportunity to work across the complete design flow—sizing transistors, verifying circuits through simulation, performing layout, and integrating multiple blocks into a functional memory system. The iterative process of debugging design errors, optimizing timing, and ensuring correct operation through simulation deepened our appreciation for the complexity of SRAM design.

Working in a team also emphasized the importance of coordination, version control, and dividing responsibilities effectively. Through consistent collaboration and problem-solving, we were able to meet the project goals and deadlines successfully.

If given more time, we would consider adding power and area analysis, more aggressive layout optimization, and additional test cases to further validate robustness under different scenarios. Overall, the project significantly improved our skills in digital circuit design, layout methodologies, and simulation-based verification.